Microsoft Defender for Endpoint

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Microsoft Defender for Endpoint 5

What you get . . . 5

What this delivers . . . 5

The standard deployment model . . . 5

Boundary . . . 6

Effort . . . 6

1. Solution Overview — Microsoft Defender for Endpoint 6

What Microsoft Defender for Endpoint is . . . 6

Why this matters . . . 7

The four lifecycle workstreams . . . 7

How it works . . . 7

What you can control . . . 8

Scope . . . 9

What is out of scope (handled elsewhere — cross-reference, don't duplicate) . . . 9

Connections and authentication used . . . 9

The standard deployment model . . . 10

Limitations (be honest with buyers) . . . 10

2. Requirements Guide — Microsoft Defender for Endpoint 10

Deploy — onboard MDE end to end . . . 11

Operate & Optimize — tune an existing deployment . . . 12

Enable — hands-on team labs . . . 13

Prove — guided proof-of-concept . . . 14

Checklist . . . 14

Global Requirements — Defender (category-level) 15

G1. Microsoft 365 / Entra tenant . . . 15

G2. Defender licensing . . . 15

G3. Administrative roles (least privilege) . . . 16

G4. Onboarding reach to endpoints / identity / workloads . . . 16

G5. Network access to Defender service endpoints . . . 16

G6. A pilot ring . . . 16

G7. Log Analytics workspace (where required) . . . 16

G8. Response governance . . . 16

3. Implementation Guide — Microsoft Defender for Endpoint 17

Naming (set once, reuse verbatim) . . . 17

Part A — Deploy (onboard MDE end to end) 18

Step A1 — Assess readiness . . . 18
Step A2 — Onboard the pilot ring . . . 18
Step A3 — Next-gen AV and EDR in block mode . . . 18
Step A4 — Attack surface reduction (audit first) . . . 18
Step A5 — Tamper protection and web content filtering . . . 19
Step A6 — Device groups and RBAC . . . 19
Step A7 — Live response and automated response (approval-gated) . . . 19
Step A8 — Validate coverage, detections, and TVM baseline on the ring . . . 19
Step A9 — Expand across the fleet . . . 20
Part B — Operate & Optimize (tune the deployment) 20
Step B1 — Baseline the current deployment (measure "before") . . . 20
Step B2 — Move ASR rules audit → block on the pilot ring . . . 20
Step B3 — Reduce alert noise and false positives (pilot) . . . 21
Step B4 — Expand coverage and standardize tags/groups . . . 21
Step B5 — Harden tamper protection, EDR block mode, and web filtering (pilot) . . . 21
Step B6 — Validate the pilot (measure "after") and expand ring by ring . . . 22
Part C — Enable (hands-on team labs) 22
Step C1 — Prepare the cohort and lab environment . . . 22
Step C2 — Run the guided labs . . . 23
Step C3 — Validate (capstone + knowledge-check) and operationalize . . . 23
Part D — Prove (guided proof-of-concept) 24
Step D1 — Assess readiness (read-only) . . . 24
Step D2 — Enable MDE on the pilot ring . . . 24
Step D3 — Generate benign test signal . . . 24
Step D4 — Validate, demonstrate, and score . . . 25
Step D5 — Tear down (clean rollback) and decide . . . 25
5. Testing Guide — Microsoft Defender for Endpoint 25
Part A — Deploy validation . . . 26
Part B — Operate & Optimize validation . . . 27
Part C — Enable validation (lab gates) . . . 28
Part D — Prove validation (PoC gates) . . . 28
6. Troubleshooting Guide — Microsoft Defender for Endpoint 29
Deploy & coverage . . . 29
Optimize & tuning . . . 31
Enable (labs) . . . 31
Prove (PoC) . . . 32

Cross-cutting . . . 32
7. Best Practices — Microsoft Defender for Endpoint 33
Deploy in rings, never all at once . . . 33
Bring ASR and block mode up audit-first — and earn every block with evidence . . . 33
Enforce tamper protection early, and harden without a split brain . . . 33
Structure device groups, tags, and RBAC deliberately . . . 34
Measure everything you optimize . . . 34
Tune noise with a scalpel, not a chainsaw . . . 34
Enable the team as a program, not a one-off class . . . 34
Prove value safely and reversibly . . . 34
Keep automated response approval-gated until trusted . . . 35
Keep licensing and preview facts current . . . 35
Defer the security decisions to the owners . . . 35
Where to go next . . . 35
8. Deliverables Summary — Microsoft Defender for Endpoint 36
Documentation . . . 36
Scripts (Microsoft Graph Security / Defender APIs + KQL) . . . 36
What the buyer receives . . . 39
Outcome . . . 39
Appendix A — PowerShell Scripts 40

Executive Summary — Microsoft Defender for Endpoint

White-label · Defender

Most organizations that own Microsoft Defender for Endpoint (MDE) run it at a fraction of its value. Devices are "onboarded" but the protection stack is half-configured — **EDR in block mode** off, **attack surface reduction (ASR)** rules unset or stuck in audit forever, **tamper protection** unenforced, **web content filtering** empty, **device groups and RBAC** flat, **live response** unconfigured, **vulnerability exposure** un-triaged. The console reports telemetry but blocks little, the alert queue is noisy, the SOC was never upskilled on the product, and no one ever proved to leadership that it actually works. This solution takes MDE **end to end, as one comprehensive lifecycle**: **deploy** it right across Windows/macOS/Linux, **operate and optimize** it (tune ASR audit→block, cut false positives, expand coverage, harden policy), **enable your team** with hands-on investigation-and-response labs, and **prove its value** with a guided, fully reversible proof-of-concept on benign test signal.

This is a complete, self-guided package written for an administrator with no prior MDE mastery. It is **pilot-first** throughout: onboard a device ring before the fleet, bring disruptive controls (ASR, block mode) up **audit-before-block**, keep automated response **approval-gated**, measure **before/after**, and never remove an existing control to "test."

What you get

- A plain-language **Solution Overview** of the whole MDE stack and the four lifecycle workstreams — Deploy, Operate & Optimize, Enable, Prove.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite across all four workstreams.
- An **Implementation Guide** of exact, ordered steps: onboard the ring (Win/macOS/Linux) → next-gen AV + EDR block mode → ASR audit→block → tamper protection + web filtering → device groups/RBAC → live response → TVM → optimization/tuning → team-enablement labs (with a KQL lab pack) → a guided, reversible PoC.
- **Eight scripts** — PowerShell (Microsoft Graph Security + Defender APIs) plus a **KQL advanced-hunting lab pack** — for readiness discovery, configuration validation, coverage/exposure reporting, a `-WhatIf`-gated ASR audit→block promotion helper, read-only hunting, a hands-on lab runner, and a fully `-WhatIf`-gated PoC (enable → benign signal → score → teardown).
- **Testing** and **Troubleshooting** guides for a confident rollout.

What this delivers

A **fully deployed, tuned, team-ready, and demonstrably-working** Defender for Endpoint: devices onboarded across all platforms, EDR in block mode, ASR enforced from proven audit data, tamper protection on, web content filtering live, device groups and RBAC in place, live response ready, exposure triaged, noise reduced with measured before/after evidence, admins/SOC trained to investigate and respond, and value proven on real-but-safe signal — the entire lifecycle in one package instead of a half-configured console.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — the same repeatable pattern applied across every workstream: assess readiness, configure the control (or lab, or PoC), pilot on a ring, validate against explicit pass/fail gates, then expand across the fleet.

Boundary

This is the **comprehensive Defender for Endpoint lifecycle solution**. It **cross-references rather than duplicates** the catalog's narrower, adjacent solutions: **"Defender for Endpoint Integration"** (the Intune connector wiring), **"Defender for Endpoint macOS Onboarding via Intune"** (a macOS-specific rollout), **Defender XDR — Unified Onboarding** (cross-pillar correlation with Identity/Office 365/Cloud Apps), the **Sentinel + Defender XDR SOC** solution (SIEM/SOAR), and **Advanced KQL Threat Hunting**. Use those where their focused scope fits; this solution is the end-to-end MDE stand-up, tune-up, enablement, and proof. Incident-response decisions, automated-response guardrails, which detections are true incidents, and licensing determinations remain the customer's security, operations, and licensing functions; this guide provides the technical configuration and the skills.

Effort

A competent administrator can complete the pilot-ring deployment and validation in a focused engagement, run the optimization pass and enablement labs with the team, and stand up (and tear down) the PoC on a controlled cadence. Rollout scales with device count and the mix of Windows/macOS/Linux onboarding channels; the enablement and PoC workstreams are reusable on every new wave and for every new analyst.

1. Solution Overview — Microsoft Defender for Endpoint

What Microsoft Defender for Endpoint is

Microsoft Defender for Endpoint (MDE) is the **endpoint detection & response (EDR)** pillar of Microsoft Defender. It combines next-generation antivirus, behavioral EDR, attack surface reduction, threat & vulnerability management, live response, and advanced hunting into one agent that runs on **Windows, macOS, and Linux** (and mobile) and reports to the unified Defender portal at `security.microsoft.com` under **Assets** → **Devices** and **Endpoints**. Fully deployed, MDE detects and blocks endpoint attacks, reduces the exploitable surface before an attack lands, shows which devices are vulnerable, lets analysts remotely investigate a compromised host, and gives hunters raw telemetry to query with KQL.

The building blocks this solution configures, tunes, teaches, and proves:

- **Onboarding** — getting each device reporting via the right channel (Intune, Group Policy, local script, or the platform-specific method for macOS/Linux).
- **Next-generation antivirus (NGAV)** and **EDR in block mode** — real-time/cloud-delivered protection, and letting EDR block malicious artifacts even when a non-Microsoft AV is primary.
- **Attack surface reduction (ASR) rules** — pre-attack hardening (block credential theft, Office child-process abuse, script/macro tricks), brought up **audit** → **block** on proven data.
- **Tamper protection** — prevents attackers (and misconfigurations) from disabling the protection stack.

- **Web content filtering** — category-based web control from the endpoint.
- **Device groups and RBAC** — scope devices for automation levels and analyst permissions.
- **Live response** — a remote investigative shell to a device.
- **Threat & vulnerability management (TVM / MDVM)** — exposure score and remediation of vulnerable software/configs.
- **Advanced hunting & custom detections** — KQL over the device tables, and turning a tuned hunt into an alert-only detection.
- **Automated investigation & response (AIR)** — automated triage that can (when permitted) remediate — kept approval-gated here.

Why this matters

Onboarding a device is only the first 20% of MDE. A tenant that stops there gets telemetry but little protection: EDR isn't blocking, the attack surface is wide open, the agent can be silently disabled, analysts have flat permissions and no live-response path, the alert queue is noisy enough that real detections get ignored, and no one on the team can confidently drive an investigation. This solution closes all of those gaps in one lifecycle: it brings the whole stack up safely, tunes it with before/after evidence, upskills the people who operate it, and proves the value with a scoped, reversible demonstration — so EDR is genuinely detecting, blocking, operable, trusted, and proven, not just "green in the portal."

The four lifecycle workstreams

This solution runs as four workstreams, each following the standard model. The Implementation Guide walks them in order; a team still evaluating MDE can run the **Prove** workstream first as a pre-purchase proof.

1. **Deploy (onboard end to end).** Onboard the pilot ring across Windows/macOS/Linux, enable next-gen AV and EDR in block mode, roll ASR audit→block, enforce tamper protection, configure web content filtering, build device groups and RBAC, enable live response, and baseline TVM exposure — then expand across the fleet.
2. **Operate & Optimize (tune an existing deployment).** Baseline the current state, move vetted ASR rules from audit to block on the evidence, cut alert noise and false positives with scoped suppressions/indicators, expand coverage and standardize device tags/groups, harden tamper/EDR/web-filter to best practice, and **measure the before/after delta**.
3. **Enable (upskill the team).** Guided hands-on labs take admins/SOC from the console to confident EDR investigation and response — incidents and device timelines, ASR audit-mode, read-only live response, response-action walkthroughs, endpoint advanced hunting with a KQL lab pack, and a hunt→alert-only detection capstone, ending in a scored knowledge-check.
4. **Prove (guided proof-of-concept).** Enable MDE on a small ring of test devices, generate **benign** test signal (EICAR + Microsoft's built-in evaluation simulations — never real malware), demonstrate that it produces a real detection/timeline/hunting row/automated investigation, score it against agreed criteria, then **tear it all down** cleanly.

How it works

1. **Onboarding** — the pilot ring is onboarded through the right channel per platform: **Intune** (default for Windows/macOS), **Group Policy / local script** for Windows where Intune isn't present, and the documented **macOS** (MDM profile + package) and **Linux** (repo + package) methods. *Cross-reference the catalog's Intune-integration and macOS-onboarding solutions for those channels; this solution drives the end-to-end stand-up.*

2. **NGAV + EDR in block mode** is enabled so Defender protects in real time and EDR blocks malicious artifacts post-breach even behind a third-party AV.

3. **ASR rules** run in **audit** first (to see what they would block), are reviewed against the deployment's own audit data, then moved to **block** for the ring — the same discipline whether deploying fresh or optimizing.

4. **Tamper protection** is enforced so the stack can't be disabled.

5. **Web content filtering** policies are created for the categories the org wants controlled (P2).

6. **Device groups** are defined with per-group **automation levels**, and **RBAC** roles scope analyst permissions; **device tags** keep policy/RBAC/reporting targeting the right machines.

7. **Live response** is enabled (approval-controlled) so responders can investigate a device remotely.

8. **TVM** exposure is baselined and the top remediations triaged.

9. **Optimization** reduces alert noise (indicators, suppressions, narrow AV exclusions), moves audit-vetted ASR to block, and captures a **before/after metrics baseline**.

10. **Enablement labs** run against real telemetry, read-only or preview-first, ending in one **alert-only** detection and **audit-mode** lab ASR rules — nothing production is changed in class.

11. **The PoC** enables MDE on test devices only, fires benign signal, validates/scores, and tears down — fully reversible.

12. **Automated response (AIR)** is kept in an **approval/simulate** posture throughout every workstream.

What you can control

• **Onboarding channel per platform** — Intune / GPO / script for Windows; MDM profile + package for macOS; repo + package for Linux.

• **Ring membership** — which devices onboard, enforce, and tune first.

• **ASR posture** — audit vs block, per rule, per ring, on the evidence.

• **Web content filtering** — which categories are blocked/audited.

• **Device groups + automation levels + tags** — how much AIR each group may do (approval-gated first) and how policy/RBAC target machines.

• **RBAC** — which analysts get read vs response permissions; **live response** approval controls.

• **Noise tuning** — which detections to suppress/allow, always scoped and time-bound.

• **TVM focus** — which vulnerabilities to remediate first.

• **Enablement scope** — which labs the cohort runs and which skills to operationalize.

• **PoC scope and criteria** — exactly which test devices onboard, which capabilities to prove, and the pass/fail bar.

Scope

• End-to-end **onboarding** across Windows/macOS/Linux and the full protection stack (NGAV, EDR block mode, ASR, tamper protection, web filtering, device groups/RBAC, live response, TVM).

• **Optimization** of an existing or freshly-deployed MDE — ASR audit→block on evidence, noise/false-positive reduction, coverage/tag expansion, policy hardening, with a measured before/after baseline.

• **Team enablement** — guided investigation-and-response labs, a KQL lab pack, an alert-only detection capstone, and a scored knowledge-check.

• A **guided, reversible proof-of-concept** on benign test signal with a scorecard and clean teardown.

What is out of scope (handled elsewhere — cross-reference, don't duplicate)

• **The Intune-connector wiring** and a **macOS-specific onboarding** — the catalog's **"Defender for Endpoint Integration"** and **"macOS Onboarding via Intune"** solutions.

• **Cross-pillar XDR onboarding** (Identity, Office 365, Cloud Apps, Cloud) — the **Defender XDR — Unified Onboarding** solution.

• **Running the SOC / SIEM / SOAR** as an operational service — the **Sentinel + Defender XDR SOC** solution; deciding what is a true incident is the security team's function.

• **A maintained, operated hunt program and the KQL language deep-dive** — the **Advanced KQL Threat Hunting** solution (this solution applies endpoint hunting; it does not run a standing hunt library).

• **Adversary emulation / red-teaming** — the PoC uses benign signal only; real-attack resilience is a separate governed engagement.

• **Regulatory, records/legal, acceptable-use, and production change-approval determinations** — deferred to the customer's compliance/legal/security/ops functions.

Connections and authentication used

• The **unified Defender portal** (`security.microsoft.com` → **Settings** → **Endpoints**, **Assets** → **Devices**, **Incidents & alerts**, **Configuration management / Endpoint security**, **Vulnerability management**, **Hunting** → **Advanced hunting**, **Evaluation & tutorials**) for onboarding, ASR/AV/tamper policy, web filtering, device groups, RBAC, live response, TVM, hunting, detections, and evaluation simulations.

• **Intune / Microsoft Endpoint Manager** (`intune.microsoft.com`) as the default onboarding and endpoint-security policy channel (GPO/script/MDM/Linux repo where used).

• **Microsoft Graph Security** and the **Microsoft 365 Defender / Defender for Endpoint** APIs (via the `Microsoft.Graph` PowerShell modules) for read-only discovery, validation, reporting, the `-WhatIf`-gated ASR helper, lab queries, PoC enable/teardown, and advanced hunting (`DeviceInfo`, `DeviceProcessEvents`, `DeviceNetworkEvents`, `DeviceEvents`, `AlertInfo`, `AlertEvidence`, `DeviceTvmSoftwareVulnerabilities`). Least-privilege scopes such as `SecurityEvents.Read.All`, `ThreatHunting.Read.All`, `DeviceManagementConfiguration.Read.All` (read) and

`DeviceManagementConfiguration.ReadWrite.All` (the gated apply paths only). Government-cloud tenants pass the matching endpoints — *verify endpoints for your cloud.*

• Least-privilege roles: **Security Reader** for read-only work, **Security Administrator** (and Intune **Endpoint Security Manager** where policy lives in Intune) for configuration, **Intune Administrator** (scoped) for the PoC pilot policy, and **Global Administrator** only for one-time first-service activation.

The standard deployment model

Assess readiness → ***Configure the control (or lab, or PoC)*** → ***Pilot on a ring*** → ***Validate against explicit pass/fail gates*** → ***Expand across the fleet.*** *Repeatable pattern* ***Assess*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand****, applied to every workstream, with ASR/block audited before enforce, optimization measured before/after, labs run read-only/preview-first, the PoC fully reversible, and automated response kept approval-gated until the SOC trusts it.*

Limitations (be honest with buyers)

• MDE is **license-gated** (Defender for Endpoint **P1/P2**; some features — full EDR, advanced hunting, live response, custom detections, web content filtering, and MDVM add-ons — need **P2**). Onboarding without the plan fails or limits features — *verify current plan-to-feature mapping and availability in your tenant.*

• **ASR and block mode are disruptive** if enforced blind — they must go **audit** → **block** on a ring first; a bad ASR rule can break a business app.

• **macOS and Linux** onboarding uses platform-specific packages/channels and has feature differences from Windows — *verify current platform support and feature parity.*

• **Coverage, TVM exposure, and hunting tables** populate over time — allow warm-up after onboarding before judging.

• **Optimization reduces noise; it does not eliminate SOC triage.** Over-suppression creates blind spots — every suppression here is justified and time-bound.

• **The PoC proves the detection pipeline, not resilience to a real adversary** — benign signal only.

• MDE improves endpoint detection and response; it does not replace patching, backup, identity hardening, network controls, or the SOC's incident-response decisions.

2. Requirements Guide — Microsoft Defender for Endpoint

Verify **every** requirement that applies to the workstreams you will run. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant, Defender licensing, admin roles, onboarding reach, network access, pilot ring, workspace, response governance) are in `global-requirements-defender.md` — do not repeat them; the items below are **specific to deploying, optimizing, enabling a team on, and proving Microsoft Defender for Endpoint**. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

The requirements are grouped by workstream. **Deploy (2.1–2.7)** applies to a fresh or partial deployment; **Operate & Optimize (2.8–2.10)** applies when tuning an existing deployment; **Enable (2.11–2.13)** applies

to the team labs; **Prove (2.14–2.16)** applies to the guided PoC. A green-field engagement runs Deploy then Optimize; an already-deployed tenant may start at Optimize; the Enable and Prove workstreams layer on top of an onboarded MDE.

Deploy — onboard MDE end to end

2.1 Defender for Endpoint activated

Description: MDE initialized for the tenant under `security.microsoft.com` → **Settings** → **Endpoints**. **Why:** Onboarding packages, ASR/AV policy, device groups, and RBAC only appear after first-time activation. **Verify:** Sign in as a Security Administrator and confirm **Settings** → **Endpoints** → **Onboarding** loads. Run `MDEFULL-Discovery.ps1`. **Implement-if-missing:** Complete the one-time MDE activation (Global Administrator required for first-service activation only).

2.2 Onboarding channel per platform

Description: A working onboarding channel for each platform in scope — **Intune** (default for Windows/macOS), **Group Policy / local script** for Windows, the **macOS** MDM profile + package, and the **Linux** repo + package. **Why:** MDE collects EDR signal only from onboarded devices; each platform onboards through its own channel. **Verify:** `MDEFULL-Discovery.ps1` reports the platform mix and current onboarding coverage; confirm the Intune MDE connector is **Enabled**. **Implement-if-missing:** Establish the channel per platform — *cross-reference the catalog's "Defender for Endpoint Integration" (Intune connector) and "Defender for Endpoint macOS Onboarding via Intune" solutions* — and target the pilot ring first.

2.3 Pilot device ring defined

Description: A named pilot device group `MDE Device Group – Pilot Ring – Pilot` (Entra/Intune) covering a representative mix of Windows/macOS/Linux. **Why:** Onboarding and the disruptive controls (ASR, block mode) are validated on a ring before the fleet (per `global-requirements-defender.md` G6). **Verify:** The group exists; `MDEFULL-Validate.ps1 -Check Onboarding` checks membership/coverage. **Implement-if-missing:** Create `MDE Device Group – Pilot Ring – Pilot` before onboarding anything that enforces.

2.4 EDR / next-gen AV baseline decision

Description: A decision on **EDR in block mode** and the antivirus posture (Defender AV primary, or third-party AV with EDR block mode). **Why:** Block mode is what lets EDR block artifacts behind a non-Microsoft AV; the posture must be agreed. **Verify:** `security.microsoft.com` → **Settings** → **Endpoints** → **Advanced features** → **EDR in block mode**; `MDEFULL-Validate.ps1 -Check EDR` reports it. **Implement-if-missing:** Decide the AV posture and plan to enable EDR in block mode after pilot onboarding.

2.5 ASR rollout scope (audit-first)

Description: The **attack surface reduction** rules to deploy and the plan to run them **audit** → **block** on the ring. **Why:** ASR rules can break line-of-business apps if enforced blind; audit-first surfaces the impact. **Verify:** `MDEFULL-Validate.ps1 -Check ASR` reports current ASR rule states; a rule list and exclusions are drafted. **Implement-if-missing:** Draft the ASR rule set and exclusions, create the policy `MDE ASR Rules – Pilot Ring – Audit`, and only later promote to `MDE ASR Rules – Pilot Ring – Block`.

2.6 Tamper protection and web content filtering scope

Description: Intent to enforce **tamper protection** tenant-wide and to configure **web content filtering** categories (P2). **Why:** Tamper protection stops the stack being disabled; web content filtering adds endpoint web control. *Web content filtering requires P2 and network protection — verify current availability.* **Verify:** `security.microsoft.com` → **Settings** → **Endpoints** → **Advanced features** (tamper protection) and **Web content filtering**; `MDEFULL-Validate.ps1 -Check Config` reports both. **Implement-if-missing:** Plan tamper protection enforcement and draft the web content filtering categories/policy `MDE Web Content Filtering – Tenant – Prod`.

2.7 Device groups, RBAC, live response, and response governance

Description: The **device group** structure (with per-group AIR automation levels), the **RBAC** role model (reader / operator), **live response** approval controls, and an agreed position on **automated investigation and response (AIR)** automation levels kept approval-gated during the pilot. **Why:** Device groups drive automation scope; RBAC scopes analyst permissions; live response is powerful and must be gated; AIR can isolate devices and remediate files, so the organization decides the guardrails (per `global-requirements-defender.md` G8). **Verify:** `security.microsoft.com` → **Settings** → **Endpoints** → **Permissions** (Roles, Device groups) and **Advanced features** → **Live Response**; `MDEFULL-Validate.ps1 -Check RBAC` reports them; a documented response policy exists. **Implement-if-missing:** Plan `MDE Device Group – Pilot Ring – Pilot`, roles `MDE Security Reader – Tenant – Prod` / `MDE Security Operator – Tenant – Prod`, and live-response approval settings; keep AIR approval-gated at first. *Incident-response and automation-guardrail decisions are the SOC/security team's — this guide covers the technical configuration.*

Operate & Optimize — tune an existing deployment

2.8 An existing, working deployment with ASR audit history

Description: Where you are tuning rather than deploying fresh: MDE is already **onboarded** with devices reporting, and the target ASR rules have run in **Audit** long enough to produce representative data on your software estate. **Why:** Optimization tunes an existing deployment; moving a rule to **Block** safely depends on knowing what it *would* have blocked — that is exactly what audit data shows. **Verify:** `security.microsoft.com` → **Assets** → **Devices** shows onboarded devices with **sensor health = Active**; **Reports** → **Attack surface reduction rules** → **Audit** shows audited events, or run `MDEFULL-Hunt.ps1 -Mode ASRAudit`. *Rule availability varies by platform/plan — verify current.* **Implement-if-missing:** Complete the Deploy workstream first; set target rules to **Audit** and let them soak before reassessing for Block.

2.9 Authoritative policy surface identified

Description: A clear answer to **where** ASR/EDR/AV/tamper policy is currently authored — Intune Endpoint security, the Defender portal's configuration management, Group Policy, or Configuration Manager. **Why:** You must change policy in the **authoritative** location; editing the wrong surface silently loses to the one that actually applies, or creates a split-brain conflict. **Verify:** `security.microsoft.com` → **Configuration management** → **Endpoint security policies** and Intune → **Endpoint security**; confirm which owns the target devices. `MDEFULL-Discovery.ps1` reports Intune endpoint-security policies. **Implement-if-missing:** Decide the single authoritative surface (Intune is recommended for cloud-managed devices) and migrate GPO/CM settings there before tuning.

2.10 A captured before-state baseline, change control, and rollback

Description: A saved snapshot of the current alert volume, false-positive proportion, ASR block coverage, and onboarding/sensor-health coverage — the "before" — plus an approved change record and a documented rollback before switching any ASR rule to block or tightening any policy. **Why:** Optimization is only credible if the improvement is **measured**; and ASR block / hardened policy can affect real applications, so the pilot-first, audit/`-WhatIf`-first approach and a written back-out are the safety net. **Verify:** A baseline file exists (`MDEFULL-Report.ps1 -SaveBaseline`); an approved change record, the pilot scope, and a written rollback (revert the ASR rule to audit / re-scope the policy) exist. **Implement-if-missing:** Run `MDEFULL-Report.ps1 -SaveBaseline` and write the rollback before making any change.

Enable — hands-on team labs

2.11 A learner cohort and a non-critical lab device group

Description: An Entra security group of the learners — `MDE Enablement Cohort – Team – Pilot` — and a small **device group** of non-critical devices, `MDE Lab Devices – Pilot Ring – Pilot`, for the live-response and ASR-audit labs. **Why:** Live response connects to a real device and ASR audit generates events; practising on dedicated, non-critical devices is the safe way (category **G6**). Do **not** use executives' or production-critical machines for the labs. **Verify:** Both groups exist and the lab devices are onboarded and healthy; `MDEFULL-Discovery.ps1` lists device groups. **Implement-if-missing:** Create both groups and add a couple of non-critical onboarded test devices to the lab group.

2.12 Plan appropriate to the labs (P1 vs P2) and split roles

Description: The plan that covers the lab features — **EDR, advanced hunting, live response, and custom detections are Plan 2 capabilities** — and a role split where learners hold a **read/hunt** role (Security Reader with hunting, or Security Operator) while the instructor holds live-response permission and the ability to author custom detections. **Why:** The investigation/hunting/live-response/detection labs depend on P2, and labs run at least privilege — only the gated demos need instructor elevation. *Verify current plan-to-feature mapping in your tenant.* **Verify:** Confirm **Advanced hunting**, **Live Response**, and **Custom detection rules** load; **Permissions & roles** show the split; **Settings** → **Endpoints** → **Advanced features** → **Live Response** is enabled. `MDEFULL-Discovery.ps1` reports advanced-features state. **Implement-if-missing:** Assign Defender for Endpoint **Plan 2** (often bundled in M365 E5 / E5 Security); assign read/hunt roles to the cohort (prefer PIM-eligible for elevated roles where Entra ID P2 is available — *verify licensing*); enable live response and grant the instructor the response role. On P1, run the prevention/ASR labs and defer the P2-only labs.

2.13 A read-only / no-production-response lab posture

Description: An agreed rule that all labs are **read-only or preview-first**: live response uses **only investigative commands**, ASR lab rules run in **audit**, the capstone detection is **alert-only**, and **no response action** (isolate, delete, disable) is executed against production during the labs. **Why:** Learners operate against live devices; the posture must be explicit so no one isolates a machine or runs a destructive command "to see what happens." **Verify:** The lab plan states the posture; ASR lab rules are audit; the detection is alert-only; live-response transcripts show only read-only commands. `MDEFULL-Validate.ps1 -Check Enablement` flags any block-mode lab rule or non-alert-only detection. **Implement-if-missing:** State it at kickoff; route any real finding to the SOC's normal incident

process (category **G8**).

Prove — guided proof-of-concept

2.14 Pilot ring of test devices only, with evaluation criteria agreed up front

Description: A small set of **non-production test devices** grouped as `MDE PoC – Pilot Ring – Pilot`, plus a short, written list of what "pass" means (e.g. *detection fires within N minutes, appears on the device timeline, is queryable in advanced hunting, triggers an automated investigation*). **Why:** The PoC must touch only disposable test devices so it can be torn down with zero production impact; and a PoC is only conclusive if the bar is set **before** you run it. **Verify:** An Entra/Intune device group `MDE PoC – Pilot Ring – Pilot` exists with test devices, and a criteria list exists; `MDEFULL-PoC.ps1 -Action Enable -WhatIf` lists membership and `MDEFULL-PoC.ps1 -Action Score` accepts the criteria file. **Implement-if-missing:** Create the group and add one to three test devices (a spare laptop or a test VM is ideal); draft 5–8 criteria with the evaluating stakeholders before enabling. *What counts as "good enough" is the customer's decision.*

2.15 A tester workstation for benign signal, with advanced-hunting read access

Description: Interactive access to a pilot device where a tester can drop the **EICAR** test file and run a Microsoft built-in simulation, plus permission to run advanced-hunting queries via the portal and the Graph Security API (`ThreatHunting.Read.All`). **Why:** The benign signal must be generated **on** an onboarded device to produce the detection; the validate and scorecard steps confirm it landed in the hunting tables (`DeviceEvents`, `AlertInfo`, `AlertEvidence`). **Verify:** You can sign in interactively to a device in `MDE PoC – Pilot Ring – Pilot`; **Hunting** → **Advanced hunting** runs a query; `MDEFULL-Validate.ps1 -Check PoC` authenticates and returns rows. **Implement-if-missing:** Ensure at least one pilot device is accessible (console, RDP, or physical); grant the running account **Security Reader** (or higher) and consent to the read-only hunting scope.

2.16 Response governance for the PoC (approval-gated)

Description: Agreement that **AIR** stays in an **approval/semi** posture for the duration of the PoC. **Why:** Even in a pilot, AIR can take actions (isolate, quarantine); the PoC proves detection without letting automation act unattended (per `global-requirements-defender.md` G8). **Verify:** AIR automation level for `MDE PoC – Pilot Ring – Pilot` is **Semi — require approval**; `MDEFULL-Validate.ps1 -Check PoC` reports it. **Implement-if-missing:** Set the pilot group's automation level to **Semi — require approval** before generating signal. *The automation-guardrail decision is the SOC/security team's.*

Checklist

Deploy

- MDE activated — `MDEFULL-Discovery.ps1`
- Onboarding channel per platform (Intune/GPO/script/macOS/Linux)
- Pilot device ring `MDE Device Group – Pilot Ring – Pilot` defined
- EDR block mode / AV posture decided
- ASR rollout scope (audit-first) drafted — `MDE ASR Rules – Pilot Ring – Audit`

- Tamper protection + web content filtering scope planned
- Device groups, RBAC, live-response, and AIR governance planned (approval-gated)

Operate & Optimize

- Existing, reporting deployment with ASR audit history — `MDEFULL-Hunt.ps1 -Mode ASRAudit`
- Authoritative policy surface identified (Intune / Defender / GPO / CM)
- Before-state baseline saved, change window + rollback documented — `MDEFULL-Report.ps1 -SaveBaseline`

Enable

- Cohort `MDE Enablement Cohort – Team – Pilot` + lab group `MDE Lab Devices – Pilot Ring – Pilot`
- Plan supports the labs (P2) and roles split (read/hunt vs instructor)
- Read-only / no-production-response lab posture agreed

Prove

- Pilot ring `MDE PoC – Pilot Ring – Pilot` (test devices only) + written criteria
- Tester workstation for benign EICAR/simulation signal + advanced-hunting read access
- AIR kept approval-gated for the PoC

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Microsoft Defender for Endpoint

Follow these steps **in order** within each workstream. Configuration spans the **unified Defender portal** (`https://security.microsoft.com`) and the **Intune admin center** (`https://intune.microsoft.com`) as the default onboarding/policy channel. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model (every workstream):*** *Assess readiness, configure on a **pilot ring** first, validate against explicit gates, then expand. Bring the disruptive controls (**ASR**, **EDR block mode**) up **audit** → **block**; keep **automated response (AIR)** approval-gated throughout; measure **before/after** when optimizing; run enablement labs **read-only/preview-first**; make the PoC **fully reversible**. Never remove an existing control to "test."*

The Implementation runs in four parts, matching the lifecycle **Deploy** → **Operate & Optimize** → **Enable** → **Prove**. A green-field tenant runs Part A then Part B; an already-deployed tenant can start at Part B. Parts C and D layer on an onboarded MDE. A team still evaluating MDE before purchase can run **Part D (Prove)** first as a pre-deployment proof, then come back to Part A.

Naming (set once, reuse verbatim)

Deploy / Optimize

- Pilot device group: `MDE Device Group – Pilot Ring – Pilot`
- Onboarding profile (Intune, Windows): `MDE Onboarding – Pilot Ring – Pilot`
- Next-gen AV / hardening policy: `MDE Next-Gen AV – Pilot Ring – Prod`, `MDE Endpoint Hardening – Pilot Ring – Pilot`
- ASR policies: `MDE ASR Rules – Pilot Ring – Audit`, `MDE ASR Rules – Pilot Ring – Block`, `MDE ASR Rules – All Devices – Prod`
- Web content filtering policy: `MDE Web Content Filtering – Tenant – Prod`
- RBAC roles: `MDE Security Reader – Tenant – Prod`, `MDE Security Operator – Tenant – Prod`
- Noise tuning (per item): `MDE FP Suppression – <detection> – Prod`, `MDE Allow Indicator – <object> – Prod`
- Advanced-hunting saved query: `MDE Onboarding Coverage – Tenant – Validate`

Enable

- Learner cohort group: `MDE Enablement Cohort – Team – Pilot`
- Lab device group: `MDE Lab Devices – Pilot Ring – Pilot`
- Shared saved-query set: `MDE Hunt Library – Team – Pilot`
- ASR lab config (audit): `MDE ASR Lab – Lab Devices – Audit`
- Custom detection (capstone, alert-only): `MDE Lab Detection – Team – Pilot`
- Knowledge-check: `MDE Knowledge Check – Team – Pilot`

Prove

- PoC pilot device group: `MDE PoC – Pilot Ring – Pilot`
- PoC Intune EDR onboarding policy: `MDE PoC EDR Onboarding – Pilot Ring – Pilot`

- Scorecard output: `MDE PoC Scorecard – Pilot Ring – Validate`
- Test-signal run tag: `MDE-PoC-TESTSIGNAL`

Part A — Deploy (onboard MDE end to end)

Step A1 — Assess readiness

1. Sign in to `security.microsoft.com` as a **Security Administrator**.

2. Run `MDEFULL-Discovery.ps1` to capture current state: MDE activation, licensed plan (P1/P2) hints, admin roles, platform mix, onboarding coverage, sensor health, endpoint-table freshness, device groups/tags, and the Intune MDE connector status.

3. In **Intune/Entra** → **Groups**, create `MDE Device Group – Pilot Ring – Pilot` with a representative Windows/macOS/Linux mix.

Step A2 — Onboard the pilot ring

1. `security.microsoft.com` → **Settings** → **Endpoints** → **Onboarding**. If MDE is not initialized, complete the one-time activation.

2. **Windows via Intune (default):** confirm `intune.microsoft.com` → **Endpoint security** → **Microsoft Defender for Endpoint** → **Connection status: Enabled**. Create the EDR onboarding profile `MDE Onboarding – Pilot Ring – Pilot` and assign it only to `MDE Device Group – Pilot Ring – Pilot`. (GPO/local-script channels use the package from **Settings** → **Endpoints** → **Onboarding** against the pilot OU.)

3. **macOS:** deploy the MDM onboarding profile + package to the pilot Macs (*cross-reference the macOS onboarding solution*). **Linux:** onboard via the documented repo + package method on the pilot Linux hosts. *Verify current macOS/Linux onboarding steps and support.*

4. On pilot devices, confirm onboarding in **Assets** → **Devices**.

5. Run `MDEFULL-Validate.ps1 -Check Onboarding` to confirm each pilot device reports EDR signal.

Step A3 — Next-gen AV and EDR in block mode

1. In Intune → **Endpoint security** → **Antivirus**, create `MDE Next-Gen AV – Pilot Ring – Prod` (real-time protection, cloud-delivered protection, PUA) assigned to the pilot ring.

2. `security.microsoft.com` → **Settings** → **Endpoints** → **Advanced features** — enable **EDR in block mode** (lets EDR block artifacts behind a third-party AV).

3. Run `MDEFULL-Validate.ps1 -Check EDR` to confirm block mode and AV posture.

Step A4 — Attack surface reduction (audit first)

1. In Intune → **Endpoint security** → **Attack surface reduction**, create `MDE ASR Rules – Pilot Ring – Audit` with the chosen ASR rules set to **Audit**, assigned to the pilot ring.

2. Let it run, then review what the rules *would* block with `MDEFULL-Report.ps1 -Section ASR` and the advanced-hunting `DeviceEvents` for `Asr*Audited`. Add exclusions for legitimate line-of-business behavior.

3. Only after review, create `MDE ASR Rules – Pilot Ring – Block` with the validated rules set to **Block**, assigned to the pilot ring.

4. **Preview the flip first:** run `MDEFULL-PromoteASR.ps1` (defaults to `-WhatIf`) to see exactly which rules would change to Block; re-run with `-Apply` (or complete it in the portal). Then run `MDEFULL-Validate.ps1 -Check ASR` to confirm rule states.

Step A5 — Tamper protection and web content filtering

1. `security.microsoft.com` → **Settings** → **Endpoints** → **Advanced features** → **Tamper Protection** — enforce it (protects the stack from being disabled).

2. **Settings** → **Endpoints** → **Web content filtering** — create `MDE Web Content Filtering – Tenant – Prod` for the categories to control (P2). *Web content filtering requires P2 and network protection — verify current availability.*

3. Run `MDEFULL-Validate.ps1 -Check Config` to confirm tamper protection and web filtering.

Step A6 — Device groups and RBAC

1. `security.microsoft.com` → **Settings** → **Endpoints** → **Permissions** → **Device groups** — define `MDE Device Group – Pilot Ring – Pilot` with the pilot ring and set its **automation level** to **Semi — require approval** during the pilot.

2. **Settings** → **Endpoints** → **Permissions** → **Roles** — create `MDE Security Reader – Tenant – Prod` (read-only) and `MDE Security Operator – Tenant – Prod` (read + response); assign least privilege.

3. Run `MDEFULL-Validate.ps1 -Check RBAC` to confirm device groups and roles.

Step A7 — Live response and automated response (approval-gated)

1. `security.microsoft.com` → **Settings** → **Endpoints** → **Advanced features** → **Live Response** — enable it, and require **live response for servers** / approval per your policy. Live response is powerful — restrict who can run it via the RBAC roles above.

2. Confirm the pilot device group's AIR **automation level** remains **Semi — require approval** (Step A6). Record it with `MDEFULL-Report.ps1`.

3. *The decision to raise automation or broaden live response is the SOC/security team's — this guide leaves it approval-gated.*

Step A8 — Validate coverage, detections, and TVM baseline on the ring

1. Run `MDEFULL-Hunt.ps1 -Mode Coverage` to execute `MDE Onboarding Coverage – Tenant – Validate` — it confirms `DeviceInfo` shows the pilot devices onboarded/healthy and that `DeviceEvents`/`DeviceTvmSoftwareVulnerabilities` are populating.

2. Run the Microsoft-provided **EDR detection test** (safe, documented) on a pilot device and confirm the alert appears in **Incidents & alerts**. Confirm an ASR-block event on a device once block mode is on.

3. `security.microsoft.com` → **Vulnerability management** → **Dashboard** — record the exposure baseline with `MDEFULL-Report.ps1 -SaveBaseline`.

4. Complete the **Testing Guide** (Tests 1–8) before expanding.

Step A9 — Expand across the fleet

1. **Onboarding:** broaden the Intune assignment (or create the next ring) to cover remaining Windows devices in waves; extend the macOS/Linux onboarding to the rest of those fleets.

2. **ASR/block mode:** promote `MDE ASR Rules – Pilot Ring – Block` to `MDE ASR Rules – All Devices – Prod` (and block mode) to production rings after each ring validates clean.

3. **Web filtering / tamper protection:** confirm tenant-wide.

4. **RBAC / automation:** review AIR automation levels with the SOC only after coverage and detections are trusted (still their decision).

5. Re-baseline exposure with `MDEFULL-Report.ps1 -SaveBaseline` and track coverage/exposure drift each cycle.

Part B — Operate & Optimize (tune the deployment)

*Baseline the current deployment, tune on the pilot ring first, **measure the before/after delta**, then expand ring by ring. Every disruptive change starts in **audit /** `-WhatIf`, and every step names a rollback. *Which detections are true incidents, and whether a suppression or ASR block is acceptable for a business-critical app, are the SOC/security team's decisions.**

Step B1 — Baseline the current deployment (measure "before")

1. `security.microsoft.com` → **Reports** → **Attack surface reduction rules** and **Assets** → **Devices**. Note current ASR rule states, onboarded/active device counts, and open-alert volume.

2. Run `MDEFULL-Discovery.ps1` to capture, read-only: onboarded devices and sensor health, device groups/tags, ASR rule states, tamper/EDR-block-mode/web-filtering state, and Intune endpoint-security policies.

3. Run `MDEFULL-Report.ps1 -SaveBaseline` to record the **before** metrics — alert volume, false-positive proportion, ASR block coverage, and onboarding/sensor-health coverage — to the baseline file.

4. Run `MDEFULL-Hunt.ps1 -Mode ASRAudit` to list which ASR rules have audit hits and on which devices (your evidence for safe block decisions).

Step B2 — Move ASR rules audit → block on the pilot ring

1. Review the ASR audit evidence: any rule with **only benign** audited events across the pilot ring is a safe **Block** candidate; any rule blocking a real line-of-business app needs an **allow/exclusion** or stays in audit. *Whether a given app is business-critical is the app owner's call.*

2. In the authoritative surface, set the vetted rules to **Block** on `MDE ASR Rules – Pilot Ring – Block` (Intune → **Endpoint security** → **Attack surface reduction**, or `security.microsoft.com` → **Configuration management** → **Endpoint security policies**); keep unproven rules at **Audit**.

3. **Preview the flip first:** run `MDEFULL-PromoteASR.ps1 -PolicyName 'MDE ASR Rules – Pilot Ring – Block' -Rules <ids>` (defaults to `-WhatIf`) to see exactly which rules would change to Block; re-run with `-Apply`.

4. **Assign** the policy to `MDE Device Group – Pilot Ring – Pilot` only. Add any exception as a scoped ASR **file/folder exclusion** on the same policy, documented — never disable the whole rule to fix one app.

Step B3 — Reduce alert noise and false positives (pilot)

1. `security.microsoft.com` → **Incidents & alerts** → **Alerts**. Identify the **repeating, benign** detections analysts already dismiss (`MDEFULL-Hunt.ps1 -Mode Noise` ranks them).

2. For a confirmed benign, recurring detection, create a **suppression rule** (Alert → **Create a suppression rule**), scoped as tightly as possible, named `MDE FP Suppression – <detection> – Prod`, with a **justification and review date**.

3. For a known-good file/cert/IP/URL, add an **allow indicator**: **Settings** → **Endpoints** → **Indicators** → **Add**, named `MDE Allow Indicator – <object> – Prod` — prefer a precise indicator over a broad AV exclusion.

4. Only where truly required, add a narrow antivirus **exclusion** in the AV policy — the last resort, because exclusions are blind spots.

5. Re-run `MDEFULL-Hunt.ps1 -Mode Noise` to confirm the tuned detections drop in volume. **Do not over-suppress** — every suppression is justified and time-bound.

Step B4 — Expand coverage and standardize tags/groups

1. Run `MDEFULL-Hunt.ps1 -Mode Coverage` (and `MDEFULL-Discovery.ps1`) to find devices **not onboarded** or whose **sensor health** is Inactive/Misconfigured.

2. Onboard the stragglers through the authoritative channel (Part A). *Truly unmanaged devices must be brought under management first.*

3. Apply a consistent **device tag** scheme (by role/site) — **Assets** → **Devices** → **(device)** → **Manage tags**, or a dynamic rule — and confirm **device groups** target correctly for policy and RBAC.

4. Run `MDEFULL-Validate.ps1 -Check Tuning` → confirms sensor health improved and tags/groups resolve.

Step B5 — Harden tamper protection, EDR block mode, and web filtering (pilot)

1. Create `MDE Endpoint Hardening – Pilot Ring – Pilot`: Intune → **Endpoint security** → **Antivirus / Endpoint detection and response** — enable **Tamper Protection**, confirm **EDR in block mode** is on, and set AV settings (cloud-delivered protection, PUA, real-time) to best practice.

2. Preview the tamper/EDR state with `MDEFULL-PromoteASR.ps1 -Mode Hardening` (reports current vs intended tamper/EDR state; changes nothing).

3. Confirm `MDE Web Content Filtering – Tenant – Prod` blocks the risk categories the security team specifies. *Web content filtering availability/plan — verify current. Choice of blocked categories is the security team's.*

4. **Assign** the hardening policy to `MDE Device Group – Pilot Ring – Pilot` only.

Step B6 — Validate the pilot (measure "after") and expand ring by ring

1. Run `MDEFULL-Validate.ps1 -Check Tuning` → confirms ASR block rules applied, tamper on, EDR block mode on, web filtering active, sensor health healthy.

2. Run `MDEFULL-Report.ps1 -BaselinePath <baseline>` → shows the **before/after delta**: lower alert/false-positive volume, higher ASR block coverage, higher onboarding/sensor-health coverage.

3. Confirm no legitimate application broke on the pilot ring. Then clone the validated pilot policies to production scope (`MDE ASR Rules – All Devices – Prod` and the hardening/web-filtering policies), roll the suppression/indicator and tag/group discipline out estate-wide, and **re-baseline** after each wave with `MDEFULL-Report.ps1 -SaveBaseline`.

Part C — Enable (hands-on team labs)

*A sequence of **guided hands-on labs** that take admins/SOC to confident EDR investigation and response. Run against real telemetry, **read-only or preview-first** — the only changes are one **alert-only** capstone detection and **audit-mode** lab ASR rules. This part **uses** the deployed MDE; it does not reconfigure it.*

Step C1 — Prepare the cohort and lab environment

1. Run `MDEFULL-Discovery.ps1` to confirm devices are reporting and endpoint tables have recent rows.

2. In **Entra** → **Groups** → **New group** create `MDE Enablement Cohort – Team – Pilot` and add learners.

3. In the Defender portal (or Intune) create `MDE Lab Devices – Pilot Ring – Pilot` with a couple of non-critical onboarded devices.

4. Assign read/hunt roles to learners; reserve live-response + detection-authoring to the instructor. Agree the **read-only / no-production-response** posture.

5. Open `MDEFULL-Labs.kql`; confirm you can paste a query into **Advanced hunting** and run it. Save each lab's good query into shared queries under `MDE Hunt Library – Team – Pilot`.

6. Note the ASR rules you will demonstrate in **audit** only as `MDE ASR Lab – Lab Devices – Audit`, scoped to `MDE Lab Devices – Pilot Ring – Pilot`. **Do not set any lab rule to block.** *Verify current ASR rule GUIDs/behavior.*

Step C2 — Run the guided labs

• **Lab 1 — Console & the EDR mental model.** Tour **Incidents & alerts**, **Assets** → **Devices**, and a device page. *Check:* navigate to any incident/device/timeline/hunting blade unaided.

• **Lab 2 — Investigate an incident (graph + alert story).** Open a real (or built-in evaluation) incident; walk the graph, expand the alert story, identify the **root cause process** and **impacted assets**. *Check:* state what started it and how far it spread.

• **Lab 3 — Read a device timeline.** On a lab device, open **Timeline**, filter a window/event types, reconstruct a sequence. *Check:* answer "what did this machine do at time X?"

• **Lab 4 — Endpoint advanced hunting.** Run the pack's **Lab 4** queries (encoded-PowerShell, process→network correlation, per-device rollup) read-only via `MDEFULL-RunLab.ps1 -Lab 4`. *Check:* find a behavior across devices and explain it.

• **Lab 5 — ASR audit first, read the events.** Set the demo rules to **audit** as `MDE ASR Lab – Lab Devices – Audit`; hunt the audit events in `DeviceEvents` (pack **Lab 5**). *Check:* read ASR audit results and reason about enforcing a rule.

• **Lab 6 — Live response (read-only only).** With the instructor (holding live response), connect to a lab device and run **only** `processes`, `dir`, `getfile` on a benign file. *Check:* understand how to pull evidence live — and why change commands stay behind approvals.

• **Lab 7 — Response actions: understand and gate.** Walk isolate / restrict / collect-package / stop-and-quarantine / run-AV-scan in the UI **without executing** against production; discuss the approval each needs and how to reverse it. *Check:* describe the right action for a scenario and its approval.

• **Lab 8 — From hunt to detection (alert-only).** Open your tuned Lab 4 query → **Create detection rule**; name it `MDE Lab Detection – Team – Pilot`, map device/account entities, set the response to **generate an alert only**. *Check:* the detection exists and is alert-only.

Step C3 — Validate (capstone + knowledge-check) and operationalize

1. **Capstone:** each learner takes an incident, establishes root cause and scope from the graph + timeline, writes a two-line summary, and names the response action they **would** recommend (not execute).

2. **Knowledge-check:** run `MDEFULL-Validate.ps1 -Check Enablement` — it walks the Testing gates (readiness, lab queries resolve, `MDE ASR Lab – Lab Devices – Audit` is audit-only, live response used read-only, `MDE Lab Detection – Team – Pilot` is alert-only) and reports a score against `MDE Knowledge Check – Team – Pilot`.

3. **Cohort report:** `MDEFULL-Report.ps1 -Section Enablement`.

4. **Operationalize deliberately:** move tuned queries from `MDE Hunt Library – Team – Pilot` into the team's standing library; advance clean lab ASR rules to block **as an operations decision** (staged, with rollback — feed this back to Part B); promote `MDE Lab Detection – Team – Pilot`

to production only if it proves quiet and accurate; use live-response change commands only through the SOC's approvals. Feed tuned hunts to the **Advanced KQL Threat Hunting** program.

Part D — Prove (guided proof-of-concept)

*Enable MDE on a **pilot ring of test devices only**, generate **benign** signal, validate/score, then **tear down**. All test signal is EICAR or a Microsoft built-in simulation — **never real malware or attacks**. Keep **AIR approval-gated** throughout. Nothing production is changed.* `MDEFULL-PoC.ps1` *runs the whole cycle via* `-Action Enable|TestSignal|Score|Teardown`*, each* `-WhatIf`*-gated where it changes anything.*

Step D1 — Assess readiness (read-only)

1. Sign in to `security.microsoft.com` as a **Security Administrator**.

2. Run `MDEFULL-PoC.ps1 -Action Enable -WhatIf` — it previews (changes nothing): the licensed MDE plan, admin roles, whether MDE is already activated, the onboarding channel, and whether `MDE PoC – Pilot Ring – Pilot` exists.

3. In **Entra** → **Groups**, create `MDE PoC – Pilot Ring – Pilot` and add one to three **test** devices only.

Step D2 — Enable MDE on the pilot ring

1. `security.microsoft.com` → **Settings** → **Endpoints** → **Onboarding**. If MDE is not yet initialized, complete the one-time activation (first time only).

2. `intune.microsoft.com` → **Endpoint security** → **Microsoft Defender for Endpoint** — confirm **Connection status: Enabled**.

3. **Endpoint security** → **Endpoint detection and response** → **Create Policy** (Windows). Name it `MDE PoC EDR Onboarding – Pilot Ring – Pilot`; set **client configuration package type = Auto from connector**; assign **only** to `MDE PoC – Pilot Ring – Pilot`.

4. Alternatively run `MDEFULL-PoC.ps1 -Action Enable` (without `-WhatIf`) to assign the EDR policy to the pilot ring — it is `-WhatIf`-gated and shows exactly what it will do first.

5. Confirm onboarding in **Assets** → **Devices**. In **Settings** → **Endpoints** → **Advanced features**, confirm **Automated Investigation** is on and set the pilot group's **automation level** to **Semi — require approval**.

Step D3 — Generate benign test signal

1. Run `MDEFULL-PoC.ps1 -Action TestSignal` from an onboarded pilot device (or follow its printed manual steps). It performs only **benign, documented** actions tagged `MDE-PoC-TESTSIGNAL`:

• Drops the **EICAR** anti-malware test file (the standard harmless string) to trigger Microsoft Defender Antivirus.

• Prints the exact portal path to launch a **Microsoft built-in EDR simulation**: `security.microsoft.com` → **Endpoints** → **Evaluation & tutorials** → **Tutorials & simulations** →

run a documented scenario against the pilot device. *These simulations are provided by Microsoft for safe evaluation — verify the current catalogue.*

1. **Do not** download real malware, run exploit tools, or disable any control. EICAR and the built-in simulations are sufficient and safe.

2. Note the time you generated each signal — you will compare against detection time in the scorecard.

Step D4 — Validate, demonstrate, and score

1. Run `MDEFULL-Validate.ps1 -Check PoC` — it confirms (read-only) the benign signal produced an **alert**, a **device timeline** entry, rows in advanced hunting (`DeviceEvents`, `AlertInfo`, `AlertEvidence`), and that an **automated investigation** started (kept approval-gated).

2. In the portal, walk the demo: open the incident → **alert story** → **device page** → **Timeline** → **Automated investigations** → an **Advanced hunting** query on the pilot device. Confirm AIR did **not** auto-act (Semi — require approval).

3. Run `MDEFULL-PoC.ps1 -Action Score -CriteriaPath .\criteria.json` — it maps each observation (detection fired, latency, timeline present, hunting visible, AIR started) to your agreed criteria and writes `MDE PoC Scorecard – Pilot Ring – Validate`. Review it with stakeholders — this is the decision artefact.

Step D5 — Tear down (clean rollback) and decide

1. Run `MDEFULL-PoC.ps1 -Action Teardown -WhatIf` first to preview every removal, then for real: **offboard** the pilot devices (**Settings** → **Endpoints** → **Offboarding**, or the Intune offboarding policy), **remove** `MDE PoC EDR Onboarding – Pilot Ring – Pilot`, **delete** the EICAR file, and **resolve** the `MDE-PoC-TESTSIGNAL` test alerts.

2. Optionally delete the pilot group; keep the scorecard file as your record.

3. If the PoC passed and you choose to proceed, move to **Part A** for a real production onboarding — do **not** scale from the PoC ring. If not proceeding, confirm teardown with `MDEFULL-Validate.ps1 -Check PoC` (it should report nothing left onboarded).

Order of operations

`[Deploy] Assess → onboard pilot ring (Win/macOS/Linux) → NGAV + EDR block mode → ASR audit → block → tamper + web filtering → device groups + RBAC → live response + AIR approval-gated → validate coverage + detections + TVM → expand → [Optimize] baseline (before) → ASR audit→block on evidence → tune noise/FPs → expand coverage + tags → harden → validate before/after → expand ring by ring → [Enable] prepare cohort → guided Labs 1–8 → capstone + knowledge-check → operationalize → [Prove] assess (-WhatIf) → enable pilot ring → benign EICAR/simulation signal → validate + demo → score → teardown (-WhatIf then real) → decide`

5. Testing Guide — Microsoft Defender for Endpoint

Run the tests for each workstream you executed. Every test has a **pass/fail gate**; do not enforce ASR/block mode more widely, expand onboarding, promote a detection, or call a PoC conclusive until the relevant gates pass. Objects are referenced by the exact names from the Implementation Guide. Scripts are referenced by name.

Part A — Deploy validation

A-Test 1 — Pilot ring onboarded (all platforms)

Procedure: Run `MDEFULL-Validate.ps1 -Check Onboarding`. In **Assets** → **Devices**, confirm each pilot device from `MDE Device Group – Pilot Ring – Pilot` (Windows/macOS/Linux) is **Onboarded**. **Pass:** Every pilot device is onboarded and reporting EDR signal. **Fail:** A device stays "Can be onboarded" — see Troubleshooting *Device not onboarding*.

A-Test 2 — Next-gen AV + EDR block mode

Procedure: Run `MDEFULL-Validate.ps1 -Check EDR`. Confirm `MDE Next-Gen AV – Pilot Ring – Prod` is applied and **EDR in block mode** is on. **Pass:** AV policy applied to the ring and block mode enabled. **Fail:** either off — see Troubleshooting *EDR/AV not applied*.

A-Test 3 — ASR in audit, then block

Procedure: Run `MDEFULL-Validate.ps1 -Check ASR` and `MDEFULL-Report.ps1 -Section ASR`. Confirm `MDE ASR Rules – Pilot Ring – Audit` produced audit events, exclusions were reviewed, and `MDE ASR Rules – Pilot Ring – Block` is blocking on the ring. **Pass:** ASR ran in audit, exclusions tuned, block enforced with no legitimate-app breakage. **Fail:** blocking before audit review, or breaking apps — revert to audit per Implementation Step A4/B2.

A-Test 4 — Tamper protection + web content filtering

Procedure: Run `MDEFULL-Validate.ps1 -Check Config`. Confirm **Tamper Protection** is enforced and `MDE Web Content Filtering – Tenant – Prod` is active. **Pass:** Tamper protection on; web filtering enforced (P2). **Fail:** either off — re-apply per Step A5; *verify P2*.

A-Test 5 — Device groups + RBAC

Procedure: Run `MDEFULL-Validate.ps1 -Check RBAC`. Confirm `MDE Device Group – Pilot Ring – Pilot` exists with automation **Semi — require approval**, and roles `MDE Security Reader – Tenant – Prod` / `MDE Security Operator – Tenant – Prod` are assigned. **Pass:** Device group and roles exist; reader cannot take response actions, operator can; automation approval-gated. **Fail:** flat permissions or wrong automation — re-apply per Step A6.

A-Test 6 — Onboarding coverage + telemetry streaming

Procedure: Run `MDEFULL-Hunt.ps1 -Mode Coverage`. Confirm `DeviceInfo` shows pilot devices onboarded/healthy and `DeviceEvents` / `DeviceTvmSoftwareVulnerabilities` return rows. **Pass:** Coverage and telemetry tables stream for the ring. **Fail:** a table empty — see Troubleshooting *No device telemetry*.

A-Test 7 — A detection actually fires

Procedure: Run the Microsoft-provided safe **EDR detection test** on a pilot device; confirm the alert appears in **Incidents & alerts**. With block mode on, confirm an ASR-block event. **Pass:** The controlled

test produces a visible MDE alert; ASR blocks the tested behavior. **Fail:** no alert/block — see Troubleshooting *Detection does not fire*.

A-Test 8 — TVM exposure baseline + AIR approval-gated

Procedure: Run `MDEFULL-Report.ps1 -SaveBaseline`. Confirm a TVM exposure baseline is recorded and the pilot group's AIR automation level is **Semi — require approval**. **Pass:** Exposure baseline captured; AIR approval-gated. **Fail:** no exposure data (allow warm-up) or automation higher than agreed — reset per Step A7 and confirm with the SOC.

Deploy exit gate: all eight pass on the pilot ring before promoting ASR/block mode or expanding onboarding (Step A9). Re-run A-Tests 1, 3, 6, 7, 8 after each expansion wave against the saved baseline.

Part B — Operate & Optimize validation

B-Test 1 — Before-state baseline captured

Procedure: Confirm the baseline from `MDEFULL-Report.ps1 -SaveBaseline` (Step B1) exists with alert volume, false-positive proportion, ASR block coverage, and onboarding/sensor-health coverage. **Pass:** A baseline exists. **Fail:** none — re-run Step B1; without a "before" there is no measurable improvement.

B-Test 2 — ASR rules blocking on the pilot ring; no legitimate app broke

Procedure: Run `MDEFULL-Validate.ps1 -Check ASR` / `-Check Tuning`. Confirm the vetted rules show **Block** on `MDE ASR Rules – Pilot Ring – Block`, assigned to `MDE Device Group – Pilot Ring – Pilot`; run `MDEFULL-Hunt.ps1 -Mode ASRAudit -BlockedOnly` and review block events for any business app. **Pass:** Intended rules block; only malicious/unwanted behavior blocked; no business-critical app broken. **Fail:** rules still audit/unassigned, or a legitimate app blocked — add a scoped exclusion or revert that rule to audit (rollback).

B-Test 3 — Alert noise / false positives reduced (measured)

Procedure: Run `MDEFULL-Report.ps1 -BaselinePath <baseline>`. Compare current alert volume and false-positive proportion to the B-Test 1 baseline. **Pass:** Volume and/or FP proportion **measurably lower** than baseline, with no new blind spot from over-suppression. **Fail:** no reduction — confirm suppressions/indicators applied and scoped.

B-Test 4 — Suppressions scoped and time-bound

Procedure: Run `MDEFULL-Validate.ps1 -Check Tuning`. In **Incidents & alerts** → **suppression rules** and **Settings** → **Endpoints** → **Indicators**, confirm each is named `MDE FP Suppression – <detection> – Prod` / `MDE Allow Indicator – <object> – Prod` with a justification and review date. **Pass:** every suppression/indicator narrow, justified, with a review date. **Fail:** a broad/permanent one exists — tighten or add an expiry.

B-Test 5 — Coverage expanded + tags/groups resolve

Procedure: Run `MDEFULL-Validate.ps1 -Check Tuning` and `MDEFULL-Hunt.ps1 -Mode Coverage`. Compare onboarded/active count to baseline; filter **Assets** → **Devices** by the applied tag. **Pass:** onboarding/sensor-health coverage **higher** than baseline; tags/groups target the right devices. **Fail:** coverage unchanged, devices Inactive, or tags missing — see Troubleshooting.

B-Test 6 — Hardening on (tamper / EDR block mode / web filtering)

Procedure: Run `MDEFULL-Validate.ps1 -Check Config`. Confirm **Tamper Protection = On**, **EDR in block mode = On**, and `MDE Web Content Filtering – Tenant – Prod` active for the ring. **Pass:** all three enabled and applied. **Fail:** one off — re-run Step B5.

Optimize exit gate: all pass on the pilot ring — with a **measurable before/after improvement** in B-Test 3 and B-Test 5 — before estate-wide expansion (Step B6). Re-run after each wave against the re-saved baseline.

Part C — Enable validation (lab gates)

C-Test 1 — Environment supports the labs

Procedure: Run `MDEFULL-Discovery.ps1`; confirm active devices and that `DeviceProcessEvents`/`DeviceNetworkEvents`/`DeviceEvents` return recent rows, and that Advanced hunting / Live Response / Custom detection rules are present (P2). **Pass:** data present and P2 surfaces load. **Fail:** onboard devices / assign P2, or run only the prevention/ASR labs on P1.

C-Test 2 — Hunting labs resolve

Procedure: `MDEFULL-RunLab.ps1 -Lab 4` returns rows; repeat for Lab 5. **Pass:** Labs 4–5 run without schema errors and return explainable results. **Fail:** correct any table/column/ASR-ActionType name to your tenant's schema and re-run.

C-Test 3 — Learners can investigate; lab ASR is audit-only; live response read-only

Procedure: Observe the capstone (root cause + scope from graph + timeline); confirm `MDE ASR Lab – Lab Devices – Audit` is **audit** (never block) with events in `DeviceEvents`; review the live-response transcript for `MDE Lab Devices – Pilot Ring – Pilot` — only `processes`/`dir`/`fileinfo`/`getfile`. `MDEFULL-Validate.ps1 -Check Enablement` flags any block-mode lab rule or change command. **Pass:** capstone identifies root cause + scope; lab ASR audit-only; only read-only live-response commands. **Fail:** reinforce the mapped lab; reaffirm the read-only posture (**R2.13**).

C-Test 4 — Lab detection is alert-only; knowledge-check passes

Procedure: Advanced hunting → **Custom detection rules** lists `MDE Lab Detection – Team – Pilot` with response = alert-only; run `MDEFULL-Validate.ps1 -Check Enablement` for the `MDE Knowledge Check – Team – Pilot` score. **Pass:** detection exists and is alert-only; each learner passes the knowledge-check. **Fail:** recreate the detection as alert-only (Lab 8); revisit the mapped lab for any missed question.

Enable exit gate: all lab gates pass and no production response action was taken; then operationalize (Step C3).

Part D — Prove validation (PoC gates)

D-Test 1 — Pilot onboarding

Procedure: Run `MDEFULL-PoC.ps1 -Action Enable -WhatIf`, then after enabling confirm each device in `MDE PoC – Pilot Ring – Pilot` shows **Onboarded** in **Assets** → **Devices**. **Pass:** every pilot device onboarded. **Fail:** a device stays "Can be onboarded" — see Troubleshooting *Pilot device not onboarding*.

D-Test 2 — Benign signal generated

Procedure: Run `MDEFULL-PoC.ps1 -Action TestSignal` on a pilot device; confirm the EICAR file was written/quarantined and a built-in simulation ran. **Pass:** Microsoft Defender Antivirus detects/quarantines EICAR; the simulation runs; only benign signal used. **Fail:** no AV reaction — see Troubleshooting *EICAR not detected*.

D-Test 3 — Detection reached the portal, timeline, and hunting

Procedure: Run `MDEFULL-Validate.ps1 -Check PoC`. Confirm an alert from the pilot device in **Incidents & alerts**, the EICAR/simulation activity on the **device timeline** with process/file detail, and recent rows in `DeviceEvents`/`AlertInfo`/`AlertEvidence`. **Pass:** alert within your agreed latency window; timeline rich; hunting rows return. **Fail:** empty — allow warm-up; see Troubleshooting *No alert from test signal* / *Timeline empty* / *Hunting tables empty*.

D-Test 4 — Automated investigation started (approval-gated)

Procedure: Open **Incidents** → **the test incident** → **Automated investigations**; `MDEFULL-Validate.ps1 -Check PoC` reports AIR status and automation level. **Pass:** an automated investigation started and the pilot group's level is **Semi — require approval** (no unattended action). **Fail:** no AIR, or automation acted unattended — see Troubleshooting *AIR did not start* / reset per Step D2.

D-Test 5 — Scorecard produced

Procedure: Run `MDEFULL-PoC.ps1 -Action Score` and confirm `MDE PoC Scorecard – Pilot Ring – Validate` is written with a pass/fail per criterion. **Pass:** a complete scorecard maps every observation to your criteria. **Fail:** missing observations — re-run the relevant validate step first.

D-Test 6 — Clean teardown

Procedure: Run `MDEFULL-PoC.ps1 -Action Teardown -WhatIf`, then for real; re-run `MDEFULL-Validate.ps1 -Check PoC`. **Pass:** pilot devices offboarded, the Intune policy removed, EICAR deleted, test alerts resolved — nothing left onboarded. **Fail:** residue remains — see Troubleshooting *Teardown left residue*.

Prove exit gate: D-Tests 1–5 pass to call the PoC conclusive; D-Test 6 passes to close it out. Keep the scorecard as the decision record.

6. Troubleshooting Guide — Microsoft Defender for Endpoint

Common field issues, in **symptom** → **cause** → **fix** form, grouped by workstream. Run `MDEFULL-Discovery.ps1` and the relevant `MDEFULL-Validate.ps1 -Check <area>` first — most causes surface there.

Deploy & coverage

Device not onboarding

Symptom: A pilot device stays "Can be onboarded" in **Assets** → **Devices** and reports no EDR signal (A-Test 1 / B-Test 5). **Cause:** The onboarding profile is not assigned, the Intune MDE connector is off, the device cannot reach the Defender service endpoints, or the platform channel (macOS profile / Linux package) did not complete. **Fix:** Confirm the device is in `MDE Device Group – Pilot Ring – Pilot` and `MDE Onboarding – Pilot Ring – Pilot` is assigned; confirm **Endpoint security** → **Microsoft Defender for Endpoint** → **Connection status: Enabled**; verify outbound HTTPS to the Defender endpoints (per `global-requirements-defender.md` G5); for macOS/Linux confirm the MDM profile/package installed. Force a sync and re-run `MDEFULL-Validate.ps1 -Check Onboarding`.

EDR/AV not applied

Symptom: A-Test 2 fails — **EDR in block mode** is off or `MDE Next-Gen AV – Pilot Ring – Prod` is not applied. **Cause:** Block mode not enabled in Advanced features, the AV policy not assigned to the ring, or a third-party AV conflicting. **Fix:** Enable **Settings** → **Endpoints** → **Advanced features** → **EDR in block mode**; confirm the AV policy is assigned to the ring; confirm the AV posture (Defender primary or passive behind third-party). Re-run `MDEFULL-Validate.ps1 -Check EDR`.

No device telemetry

Symptom: A-Test 6 fails — `DeviceInfo`, `DeviceEvents`, or `DeviceTvmSoftwareVulnerabilities` return no rows. **Cause:** Devices newly onboarded and still warming up, or a device unhealthy/not reporting. **Fix:** Confirm devices are onboarded and healthy (A-Test 1); allow warm-up (tables populate over hours); check device health in **Assets** → **Devices**. Re-run `MDEFULL-Hunt.ps1 -Mode Coverage`.

Detection does not fire

Symptom: A-Test 7 fails — the safe EDR detection test produces no alert. **Cause:** The device is not fully onboarded, block mode/AV is off, or not enough time has passed. **Fix:** Confirm onboarding (A-Test 1) and AV/block mode (A-Test 2); run the Microsoft-documented EDR test exactly; allow the alert to surface in **Incidents & alerts**. Re-check with `MDEFULL-Hunt.ps1 -Mode Coverage`.

Web content filtering not enforcing

Symptom: A-Test 4 / B-Test 6 fails — categories in `MDE Web Content Filtering – Tenant – Prod` are not blocked. **Cause:** Web content filtering requires **P2** and network protection in block mode; the plan or dependency is missing. **Fix:** *Verify P2 availability in your tenant*; confirm network protection is enabled (web filtering depends on it); allow policy propagation. Re-run `MDEFULL-Validate.ps1 -Check Config`.

RBAC / device group not effective

Symptom: A-Test 5 fails — analysts have inconsistent access or the automation level is wrong. **Cause:** Roles not assigned, device group scope wrong, or RBAC not yet propagated. **Fix:** Confirm `MDE Security Reader – Tenant – Prod` / `MDE Security Operator – Tenant – Prod` assignments and the `MDE Device Group – Pilot Ring – Pilot` scope/automation level under **Settings** → **Endpoints** → **Permissions**; allow propagation and re-test. Re-run `MDEFULL-Validate.ps1 -Check RBAC`.

Optimize & tuning

ASR rule not blocking / breaking a business app

Symptom: B-Test 2 fails — a vetted rule still shows Audit/Not configured, or a line-of-business app fails after moving to block. **Cause:** The rule was not applied to `MDE ASR Rules – Pilot Ring – Block` / not assigned, or a rule blocks a legitimate behavior with no exclusion. **Fix:** Confirm the rule state and assignment (re-run `MDEFULL-PromoteASR.ps1` in `-WhatIf`, then `-Apply`); for a broken app, identify the rule from the block event (`MDEFULL-Hunt.ps1 -Mode ASRAudit -BlockedOnly`), revert that rule to **audit**, add a scoped exclusion, validate in audit, then re-promote. This is exactly why ASR goes **audit** → **block** first.

No baseline / report empty

Symptom: B-Test 1/3 fails — no before-state metrics to compare against. **Cause:** `MDEFULL-Report.ps1 -SaveBaseline` was never run, or the deployment had no telemetry when it ran. **Fix:** Run `MDEFULL-Report.ps1 -SaveBaseline` on a reporting deployment before any change; without a "before" there is no measurable improvement.

Noise not dropping / over-broad suppression

Symptom: B-Test 3/4 fails — alert volume unchanged, or a suppression is too broad/permanent. **Cause:** Suppressions/indicators not applied, scoped too widely, or missing a review date. **Fix:** Confirm each `MDE FP Suppression – <detection> – Prod` / `MDE Allow Indicator – <object> – Prod` is applied, tightly scoped, justified, and time-bound; tighten or add an expiry. Re-run `MDEFULL-Hunt.ps1 -Mode Noise` and `MDEFULL-Validate.ps1 -Check Tuning`.

Tags/groups not resolving / split-brain policy

Symptom: B-Test 5 fails — tags missing, a group is empty, or a setting you changed did not take effect. **Cause:** The tag scheme/dynamic rule is wrong, or the same setting is authored in two surfaces (Intune and GPO/CM) and the non-authoritative one is silently losing. **Fix:** Confirm the tag scheme (Step B4) and that policy is authored only in the authoritative surface (Requirement 2.9); migrate stray GPO/CM settings into Intune. Allow propagation and re-run `MDEFULL-Validate.ps1 -Check Tuning`.

Enable (labs)

Hunting lab query errors

Symptom: C-Test 2 fails — a lab query from `MDEFULL-Labs.kql` throws a schema error or returns nothing. **Cause:** Table/column/ASR-ActionType names differ in your tenant/cloud, or the lookback window is too narrow. **Fix:** Correct the flagged `// VERIFY` names against **Advanced hunting** → **schema**; widen `-LookbackDays`; confirm table freshness with `MDEFULL-Discovery.ps1`. Re-run `MDEFULL-RunLab.ps1 -Lab <n>`.

Lab ASR rule blocked, or a change command was run

Symptom: C-Test 3 fails — `MDE ASR Lab – Lab Devices – Audit` shows a rule in block, or the live-response transcript shows `run`/`remediate`/`putfile`. **Cause:** The read-only / no-production-response posture (Requirement 2.13) was not held. **Fix:** Set the lab rules back to **audit** (they never blocked anything if audit-only); reaffirm that change commands are out of scope for the labs — reserve them for real incidents under the SOC's approvals.

Lab detection isn't alert-only

Symptom: C-Test 4 fails — `MDE Lab Detection – Team – Pilot` has a response action beyond alerting. **Cause:** The detection was created (Lab 8) with an automatic isolation/remediation response. **Fix:** Edit the custom detection so the response is **generate an alert only**; disable or delete it if it is no longer needed. `MDEFULL-Validate.ps1 -Check Enablement` confirms it is alert-only.

Prove (PoC)

Pilot device not onboarding

Symptom: D-Test 1 fails — a device in `MDE PoC – Pilot Ring – Pilot` stays "Can be onboarded." **Cause:** `MDE PoC EDR Onboarding – Pilot Ring – Pilot` is not assigned, the MDE-Intune connector is off, or the device cannot reach the Defender service endpoints. **Fix:** Confirm the device is in the pilot group; confirm **Connection status: Enabled**; verify outbound HTTPS to the Defender endpoints (G5). Force an Intune sync and re-run `MDEFULL-PoC.ps1 -Action Enable -WhatIf`.

EICAR not detected

Symptom: D-Test 2 fails — the EICAR test file is written but Microsoft Defender Antivirus does not detect/quarantine it. **Cause:** Real-time protection is off, third-party AV is primary, tamper/exclusions suppress it, or the file was altered. **Fix:** Confirm real-time and cloud-delivered protection are on for the pilot device; remove any test-path exclusion; ensure the EICAR string is exact and unmodified. Re-run `MDEFULL-PoC.ps1 -Action TestSignal`.

No alert from test signal / timeline empty / hunting tables empty

Symptom: D-Test 3 fails — no alert, an empty **Timeline**, or no rows from `DeviceEvents`/`AlertInfo`/`AlertEvidence`. **Cause:** The device is not fully onboarded, signal has not propagated, onboarding is too recent, or the account lacks hunting read scope. **Fix:** Confirm D-Test 1 passed; allow warm-up (minutes to hours); confirm the simulation/EICAR ran on an onboarded pilot device; confirm `ThreatHunting.Read.All` consent and **Security Reader**. Re-run `MDEFULL-Validate.ps1 -Check PoC`.

AIR did not start

Symptom: D-Test 4 fails — no automated investigation for the test incident. **Cause:** Automated Investigation is off, the device is not in an automation-enabled group, or the alert type did not trigger AIR. **Fix:** Confirm **Settings** → **Endpoints** → **Advanced features** → **Automated Investigation** is on and the pilot group's level is **Semi — require approval**; some benign simulations trigger AIR, some do not — use a built-in EDR simulation known to raise an EDR alert. Re-run `MDEFULL-Validate.ps1 -Check PoC`.

Teardown left residue

Symptom: D-Test 6 fails — after `MDEFULL-PoC.ps1 -Action Teardown`, a device still shows onboarded or the Intune policy remains. **Cause:** Offboarding is asynchronous, or teardown ran in `-WhatIf` only. **Fix:** Run teardown **without** `-WhatIf` to apply; offboarding can take time to reflect in **Assets** → **Devices** — allow it, then confirm with `MDEFULL-Validate.ps1 -Check PoC`. Remove the Intune policy and pilot group manually if any remain.

Cross-cutting

Automated response acted without approval

Symptom: A pilot/test device was isolated or a file remediated automatically (any workstream). **Cause:** The device group's AIR **automation level** is higher than **Semi — require approval**. **Fix:** Reset the automation level to **Semi — require approval**, confirm with the SOC, and record it with `MDEFULL-Report.ps1`. *The automation-guardrail decision is the security team's.*

Script cannot authenticate

Symptom: An `MDEFULL-*.ps1` script fails on connect or returns access-denied. **Cause:** The Microsoft Graph modules are missing, consent to the required read scopes was not granted, the account lacks the role, or (Government cloud) the wrong environment/endpoints were used. **Fix:** Let the script install the modules (it checks on run); sign in with `Connect-MgGraph` and consent to the read-only scopes (or the write scope for the gated apply paths); confirm **Security Reader** (or Security Administrator / Endpoint Security Manager for change paths); for Government cloud pass the matching `-Environment`. Re-run the script.

7. Best Practices — Microsoft Defender for Endpoint

Field-proven guidance for keeping the Defender for Endpoint lifecycle protective, operable, measurable, and low-risk — across all four workstreams.

Deploy in rings, never all at once

- Onboard `MDE Device Group – Pilot Ring – Pilot` first — a representative Windows/macOS/Linux mix — validate, then expand in waves. Sequencing catches platform-specific issues before they hit the fleet.
- Onboard **every** platform in scope; a Linux or macOS fleet left un-onboarded is a blind spot.

Bring ASR and block mode up audit-first — and earn every block with evidence

- Always run **ASR** in `MDE ASR Rules – Pilot Ring – Audit` first, review what it *would* block (`MDEFULL-Report.ps1 -Section ASR` / `MDEFULL-Hunt.ps1 -Mode ASRAudit`), add exclusions for legitimate line-of-business behavior, then promote to `MDE ASR Rules – Pilot Ring – Block`. Never flip a rule straight to block.
- Preview every flip with `MDEFULL-PromoteASR.ps1` (`-WhatIf` default) before applying, and promote ring-by-ring, validating each ring clean before the next.
- Fix a single blocked app with a **scoped exclusion**, not by disabling the whole rule. Whether an app is business-critical is the app owner's call.

Enforce tamper protection early, and harden without a split brain

- Turn on **tamper protection** so an attacker (or misconfiguration) cannot disable the protection stack — one of the highest-value, lowest-friction controls. Keep **EDR in block mode** on.

• Author ASR/EDR/AV/tamper policy in **one** authoritative surface (Intune recommended for cloud-managed devices). Editing GPO and Intune for the same setting creates conflicts that silently win in ways you didn't intend.

Structure device groups, tags, and RBAC deliberately

• Use **device groups** to scope AIR automation levels and analyst access; keep the pilot at **Semi — require approval**. Grant least privilege via `MDE Security Reader – Tenant – Prod` and `MDE Security Operator – Tenant – Prod` — only responders get response permissions.

• Standardize **device tags and groups** so policy, RBAC, and reporting target the right machines — inconsistent tagging is why "we thought it was covered" incidents happen.

• **Live response** is powerful (remote shell) — restrict who can run it and gate it per policy.

Measure everything you optimize

• Capture the **before** state (`MDEFULL-Report.ps1 -SaveBaseline`) before you touch anything; optimization you cannot measure is just change. Re-baseline after each wave so the trend reflects real scope growth.

• Work **TVM exposure as a trend** — triage the highest-impact vulnerabilities first and track the delta each cycle.

Tune noise with a scalpel, not a chainsaw

• Prefer a **precise indicator** or a tightly scoped **suppression** over a broad AV exclusion — exclusions are blind spots an attacker can hide in.

• Every suppression/indicator is **named, justified, and time-bound** (`MDE FP Suppression – <detection> – Prod`). Review them on a cadence so they don't become permanent gaps. The goal is a queue where a remaining alert **means something** — not zero alerts.

Enable the team as a program, not a one-off class

• **Investigate, don't just triage:** before closing an alert, walk the **incident graph** and **alert story** and open the **device timeline** to establish root cause and blast radius. Re-hunt every new indicator across all devices, not just the one that alerted.

• **Read-only live response by default** (`processes`, `dir`, `fileinfo`, `getfile`); reserve change commands for real incidents under the SOC's approvals — never as a lab reflex. Match the response action to the scenario and know how to reverse it.

• **Alert-only first** for detections: promote a hunt to `MDE Lab Detection – Team – Pilot` only once it is quiet and accurate. A noisy detection trains the team to ignore alerts. Version your endpoint KQL library in source control.

• Schedule recurring investigation drills, rotate who leads the hunt of the week, and re-run the take-home runbook + knowledge-check for new hires.

Prove value safely and reversibly

• Onboard **test devices only** into `MDE PoC – Pilot Ring – Pilot`; never point a PoC at production endpoints. Preview every change with `-WhatIf` first.

• Use **EICAR** and Microsoft's **built-in evaluation simulations** exclusively — enough to prove the detection pipeline. Do **not** download live malware or run exploit frameworks; adversary emulation belongs in a separate, governed engagement.

• Agree the **evaluation criteria in writing first**; a PoC that defines "pass" afterwards proves nothing. Allow warm-up before judging a "fail," then **tear down completely** and confirm nothing is left onboarded.

Keep automated response approval-gated until trusted

• Leave **AIR** at **Semi — require approval** during deployment, tuning, labs, and the PoC. Raising automation is the SOC/security team's decision, made only after coverage and detections are proven (per `global-requirements-defender.md` G8).

Keep licensing and preview facts current

• MDE **P1/P2**, MDVM add-ons, web content filtering, advanced hunting, live response, custom detections, and device-tagging behavior are plan-gated, and Microsoft's simulation catalogue and ASR rule GUIDs change. **Verify current plan requirements and availability in your tenant** before relying on a capability.

Defer the security decisions to the owners

• Which detections are **true incidents**, whether a **suppression or residual risk** is acceptable, which web categories to block, which ASR rules to enforce in production, whether the PoC "passed," and every production response are the **SOC/security team's** determinations. This solution delivers the controls, the skills, and the before/after evidence — not the incident-response or risk decision.

Where to go next

Once MDE is deployed, tuned, team-ready, and proven, the natural next steps in the catalog:

• The **Defender for Endpoint Integration** solution for deeper Intune-connector configuration, and the **macOS Onboarding via Intune** solution for a macOS-specific rollout — cross-reference rather than re-do.

• The **Microsoft Defender XDR — Unified Onboarding** solution to correlate MDE with Identity, Office 365, and Cloud Apps in one incident view.

• The **SOC onboarding — Sentinel + Defender XDR** solution to stream MDE into a SIEM for long-term hunting and SOAR.

• The **Advanced KQL Threat Hunting** solution to adopt and operate a curated, maintained hunt program built on the endpoint hunting skill this solution gives you.

• The **Microsoft Defender for Cloud Apps** — MDE's discovery feed powers Cloud Discovery.

8. Deliverables Summary — Microsoft Defender for Endpoint

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box — the whole MDE lifecycle in one package
1. Solution Overview	What MDE is; the four lifecycle workstreams (Deploy, Operate & Optimize, Enable, Prove); scope, out-of-scope cross-references, connections, limitations
2. Requirements Guide	Solution-specific prerequisites per workstream with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps across all four parts — deploy the stack, optimize with before/after, run the enablement labs, and run+tear-down the PoC
5. Testing Guide	Pass/fail gates for each workstream (Deploy 8, Optimize 6, Enable 4, Prove 6)
6. Troubleshooting Guide	Field issues and remediation across the lifecycle
7. Best Practices	Field-proven recommendations and where to go next

Scripts (Microsoft Graph Security / Defender APIs + KQL)

Script	What it does								
`MDEFULL-Discovery.ps1`	Read-only readiness across the lifecycle — MDE activation, plan (P1/P2) hints, admin roles, platform mix, onboarding coverage, sensor health, endpoint-table freshness, device groups/tags, ASR/tamper/EDR/web-filter state hints, and Intune endpoint-security policies								

Script	What it does								
`MDEFULL-Validate.ps1`	Targeted validation (`-Check Onboarding\	EDR\	ASR\	Config\	RBAC\	Tuning\	Enablement\	PoC\	All`) — confirms deploy, optimize, enablement-lab, and PoC gates from the Testing Guide
`MDEFULL-Report.ps1`	Coverage + ASR-audit + TVM exposure report and the optimization **before/after** baseline (`-Section ASR\	Exposure\	Noise\	Coverage\	Enablement\	All, -SaveBaseline, -BaselinePath`); reports AIR automation level			
`MDEFULL-PromoteASR.ps1`	The single `-WhatIf`-gated change helper — promotes audit-vetted ASR rules to **Block** on the pilot ring (`-Apply` to enforce); `-Mode Hardening` reports tamper/EDR-block-mode state read-only as a pre-change check								

Script	What it does								
`MDEFULL-Hunt.ps1`	Read-only KQL advanced-hunting (`-Mode Coverage\	ASRAudit\	Noise\	Telemetry`) — onboarding/health coverage, ASR audit vs block evidence, noisiest detections, and telemetry/TVM presence					
`MDEFULL-RunLab.ps1`	Enablement lab runner — executes a chosen endpoint hunting-lab query from the KQL pack read-only so the whole cohort can compare results								
`MDEFULL-Labs.kql`	The endpoint **KQL lab pack** — one labeled section per hunting lab (fundamentals, process→network correlation, per-device rollup, ASR audit events, logon anomalies) plus the alert-only capstone candidate; the take-home query library								

Script	What it does								
`MDEFULL-PoC.ps1`	The guided proof-of-concept in one `-WhatIf`-gated script (`-Action Enable\	TestSignal\	Score\	Teardown`) — enable MDE on a test-device ring, generate benign EICAR/simulation signal, score against your criteria, and tear it all down cleanly					

All PowerShell defaults to **read-only**; the only scripts that can change configuration — `MDEFULL-PromoteASR.ps1` and `MDEFULL-PoC.ps1` (Enable/Teardown) — default to a **safe preview / `-WhatIf`** and change nothing until explicitly confirmed.

What the buyer receives

- This solution as a branded **Word** document and **PDF**.
- All eight **scripts** (seven PowerShell + the KQL lab pack) in a single ZIP — yours to keep and reuse on every wave, cohort, and PoC.
- A repeatable process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** across the full MDE lifecycle — deploy audited-before-enforce, optimize measured before/after, enable read-only/preview-first, prove reversibly, with automated response kept approval-gated until trusted.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A **fully deployed, tuned, team-ready, and demonstrably-working** Defender for Endpoint — devices onboarded across Windows/macOS/Linux, EDR in block mode, ASR enforced on proven audit data, tamper protection on, web content filtering live, device groups and RBAC in place, live response ready, exposure triaged, alert noise reduced with measured evidence, admins/SOC trained to investigate and respond, and value proven on real-but-safe signal — the entire lifecycle in one comprehensive package.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

MDEFULL-Discovery.ps1

Purpose: Read-only discovery for the comprehensive "Microsoft Defender for Endpoint" solution. Reports the signed-in context, licensed MDE plan (P1/P2) hints, admin roles, the managed-device platform mix, current onboarding coverage and sensor health, endpoint-table freshness (for the enablement labs), device groups, and reminders for the portal-only checks (Intune MDE connector, tamper/EDR/web-filter state, endpoint-security policy surface). Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDEFULL-Discovery.ps1 Requires the Microsoft.Graph modules (installed on first run). Connects read-only with Connect-MgGraph and prompts for consent to read scopes. Government cloud: pass -Environment. EXPECTED RESULTS A readiness picture — tenant/context, plan hints, admin roles, platform mix, onboarding coverage, endpoint-table freshness, and the manual checks to confirm — written to a JSON snapshot. The state to confirm before deploying, tuning, teaching, or proving. .NOTES White-label. Read-only. Reports on configuration/identifiers only; makes no changes. Licensing/plan facts are indicative — verify current availability (P1 vs P2, MDVM) in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDEFULL-discovery.json", [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\MDEFULL-Discovery.ps1 Requires the Microsoft.Graph modules (installed on first run). Connects read-only with Connect-MgGraph and prompts for consent to read scopes. Government cloud: pass -Environment. EXPECTED RESULTS A readiness picture — tenant/context, plan hints, admin roles, platform mix, onboarding coverage, endpoint-table freshness, and the manual checks to confirm — written to a JSON snapshot. The state to confirm before deploying, tuning, teaching, or proving. .NOTES White-label. Read-only. Reports on configuration/identifiers only; makes no changes. Licensing/plan facts are indicative — verify current availability (P1 vs P2, MDVM) in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDEFULL-discovery.json", [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

MDEFULL-Hunt.ps1

Purpose: Read-only hunting for the comprehensive "Microsoft Defender for Endpoint" solution. Runs one of four saved KQL queries via the Microsoft 365 advanced-hunting API (Microsoft Graph Security). It provides the evidence behind onboarding validation (Deploy) and every tuning decision (Optimize), and makes no changes: -Mode Coverage 'MDE Onboarding Coverage - Tenant - Validate' (onboarding + health by OS; coverage gaps) -Mode ASRAudit 'ASR Audit vs Block Evidence - Tenant - Assess' (which rules would/do block) -Mode Noise 'Repeating Benign Detections - Tenant - Assess' (noisiest alert titles) -Mode Telemetry 'Endpoint Telemetry + TVM Presence - Tenant - Validate' (tables streaming?) HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDEFULL-Hunt.ps1 -Mode Coverage powershell -ExecutionPolicy Bypass -File .\MDEFULL-Hunt.ps1 -Mode ASRAudit powershell -ExecutionPolicy Bypass -File .\MDEFULL-Hunt.ps1 -Mode ASRAudit -BlockedOnly powershell -ExecutionPolicy Bypass -File .\MDEFULL-Hunt.ps1 -Mode Noise -LookbackDays 7 Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. Government cloud: -Environment. EXPECTED RESULTS Coverage: onboarding/health by OS + stale/not-onboarded devices — onboarding/sensor-health gaps. ASRAudit: per-rule audited (would-block) vs blocked counts. Noise: the highest-volume repeating detections — candidates to tune/suppress. Telemetry: recent DeviceEvents/DeviceNetworkEvents volume + TVM row

presence. .NOTES White-label. Read-only. The embedded KQL only reads telemetry. Queries are also saved in the portal under the names above. Defender XDR tables use `Timestamp`; ASR ActionType names evolve — verify. > [CmdletBinding()] param([ValidateSet('Coverage','ASRAudit','Noise','Telemetry')] [string]$Mode = 'Coverage', [int]$LookbackDays = 7, [switch]$BlockedOnly, [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\MDEFULL-Hunt.ps1 -Mode Coverage powershell -ExecutionPolicy Bypass -File .\MDEFULL-Hunt.ps1 -Mode ASRAudit powershell -ExecutionPolicy Bypass -File .\MDEFULL-Hunt.ps1 -Mode ASRAudit -BlockedOnly powershell -ExecutionPolicy Bypass -File .\MDEFULL-Hunt.ps1 -Mode Noise -LookbackDays 7 Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. Government cloud: -Environment. EXPECTED RESULTS Coverage: onboarding/health by OS + stale/not-onboarded devices — onboarding/sensor-health gaps. ASRAudit: per-rule audited (would-block) vs blocked counts. Noise: the highest-volume repeating detections — candidates to tune/suppress. Telemetry: recent DeviceEvents/DeviceNetworkEvents volume + TVM row presence. .NOTES White-label. Read-only. The embedded KQL only reads telemetry. Queries are also saved in the portal under the names above. Defender XDR tables use `Timestamp`; ASR ActionType names evolve — verify. > [CmdletBinding()] param([ValidateSet('Coverage','ASRAudit','Noise','Telemetry')] [string]$Mode = 'Coverage', [int]$LookbackDays = 7, [switch]$BlockedOnly, [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

MDEFULL-Labs.kql

Purpose: // The runnable, progressive endpoint KQL library for the enablement workstream. One labeled // section per hunting lab (endpoint fundamentals, process->network correlation, per-device // rollup, ASR audit events, logon anomalies) plus the alert-only capstone candidate. A core // take-home artifact — keep it and grow it into the team's shared query library // ('MDE Hunt Library - Team - Pilot'). // HOW TO RUN // Paste one query block (below a "// Lab N" header) into: // Microsoft Defender -> Hunting -> Advanced hunting // Or run one read-only from PowerShell: MDEFULL-RunLab.ps1 -Lab <n> // EXPECTED RESULTS // Each query returns a small, explainable result set. Columns/tables flagged // VERIFY may // differ in your tenant/cloud — correct against Advanced hunting -> schema. // NOTES // White-label. READ-ONLY — every query only reads endpoint telemetry; nothing here changes a // device. Defender XDR tables use `Timestamp`. Table/column names and ASR rule GUIDs evolve and // can differ by plan (P1/P2) and cloud — VERIFY in your tenant. // ========== ==========

How to run: // Paste one query block (below a "// Lab N" header) into: // Microsoft Defender -> Hunting -> Advanced hunting // Or run one read-only from PowerShell: MDEFULL-RunLab.ps1 -Lab <n> // EXPECTED RESULTS // Each query returns a small, explainable result set. Columns/tables flagged // VERIFY may // differ in your tenant/cloud — correct against Advanced hunting -> schema. // NOTES // White-label. READ-ONLY — every query only reads endpoint telemetry; nothing here changes a // device. Defender XDR tables use `Timestamp`. Table/column names and ASR rule GUIDs evolve and // can differ by plan (P1/P2) and cloud — VERIFY in your tenant. // ========== ==========

MDEFULL-PoC.ps1

Purpose: The Prove-workstream driver for the comprehensive "Microsoft Defender for Endpoint" solution. Runs a safe, fully reversible PoC on a small pilot ring of TEST DEVICES ONLY: -Action Enable Preview readiness, then assign the Intune EDR onboarding policy to the pilot group. Gated behind -WhatIf

(preview) — re-run without -WhatIf to apply. -Action TestSignal Generate BENIGN test signal on an onboarded pilot device: the EICAR anti-malware test file (a documented harmless string — NOT malware) plus the exact portal path for a Microsoft built-in evaluation simulation. NO real malware, exploit, or attack tooling is ever used. -Action Score Read-only: map the observed detections (alert, evidence, timeline, AIR) to your written pass/fail criteria and write the scorecard file. -Action Teardown Reverse everything the PoC created (clear/optionally delete the EDR policy, delete the local EICAR file, print offboarding + alert-cleanup steps). Gated behind -WhatIf. Validation of the detections is MDEFULL-Validate.ps1 -Check PoC. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDEFULL-PoC.ps1 -Action Enable -WhatIf powershell -ExecutionPolicy Bypass -File .\MDEFULL-PoC.ps1 -Action Enable powershell -ExecutionPolicy Bypass -File .\MDEFULL-PoC.ps1 -Action TestSignal powershell -ExecutionPolicy Bypass -File .\MDEFULL-PoC.ps1 -Action Score -CriteriaPath .\criteria.json powershell -ExecutionPolicy Bypass -File .\MDEFULL-PoC.ps1 -Action Teardown -WhatIf powershell -ExecutionPolicy Bypass -File .\MDEFULL-PoC.ps1 -Action Teardown -DeletePolicy Requires Microsoft.Graph.Authentication. Government cloud: -Environment. EXPECTED RESULTS Enable: readiness summary, then (when applied) the pilot EDR policy assigned so test devices onboard. TestSignal: EICAR written (AV should quarantine it) + printed built-in-simulation steps. Score: a per-criterion PASS/FAIL scorecard ('MDE PoC Scorecard - Pilot Ring - Validate.json'). Teardown: assignment cleared, EICAR deleted, printed offboarding + alert-resolution steps. .NOTES White-label. Read-only by default (-WhatIf) for Enable/Teardown; Score is read-only. BENIGN signal only (EICAR + Microsoft built-in simulations). Pilot/test devices only. AIR stays approval-gated. Verify the current built-in simulation catalogue and P1/P2/MDVM availability in your tenant. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)] [ValidateSet('Enable','TestSignal','Score','Teardown')] [string]$Action, [string]$PilotGroup = 'MDE PoC - Pilot Ring - Pilot', [string]$EdrPolicyName = 'MDE PoC EDR Onboarding - Pilot Ring - Pilot', [string]$Tag = 'MDE-PoC-TESTSIGNAL', [string]$WorkDir = "$env:TEMP\MDE-PoC", [string]$CriteriaPath, [string]$OutFile = 'MDE PoC Scorecard - Pilot Ring - Validate.json', [string]$DeviceName, [int]$LookbackHours = 12, [switch]$DeletePolicy, [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\MDEFULL-PoC.ps1 -Action Enable -WhatIf powershell -ExecutionPolicy Bypass -File .\MDEFULL-PoC.ps1 -Action Enable powershell -ExecutionPolicy Bypass -File .\MDEFULL-PoC.ps1 -Action TestSignal powershell -ExecutionPolicy Bypass -File .\MDEFULL-PoC.ps1 -Action Score -CriteriaPath .\criteria.json powershell -ExecutionPolicy Bypass -File .\MDEFULL-PoC.ps1 -Action Teardown -WhatIf powershell -ExecutionPolicy Bypass -File .\MDEFULL-PoC.ps1 -Action Teardown -DeletePolicy Requires Microsoft.Graph.Authentication. Government cloud: -Environment. EXPECTED RESULTS Enable: readiness summary, then (when applied) the pilot EDR policy assigned so test devices onboard. TestSignal: EICAR written (AV should quarantine it) + printed built-in-simulation steps. Score: a per-criterion PASS/FAIL scorecard ('MDE PoC Scorecard - Pilot Ring - Validate.json'). Teardown: assignment cleared, EICAR deleted, printed offboarding + alert-resolution steps. .NOTES White-label. Read-only by default (-WhatIf) for Enable/Teardown; Score is read-only. BENIGN signal only (EICAR + Microsoft built-in simulations). Pilot/test devices only. AIR stays approval-gated. Verify the current built-in simulation catalogue and P1/P2/MDVM availability in your tenant. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)] [ValidateSet('Enable','TestSignal','Score','Teardown')] [string]$Action, [string]$PilotGroup = 'MDE PoC - Pilot Ring - Pilot', [string]$EdrPolicyName = 'MDE PoC EDR Onboarding - Pilot Ring - Pilot', [string]$Tag = 'MDE-PoC-TESTSIGNAL', [string]$WorkDir = "$env:TEMP\MDE-PoC", [string]$CriteriaPath, [string]$OutFile = 'MDE PoC Scorecard - Pilot Ring - Validate.json', [string]$DeviceName, [int]$LookbackHours = 12, [switch]$DeletePolicy, [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

MDEFULL-PromoteASR.ps1

Purpose: The single change helper for the comprehensive "Microsoft Defender for Endpoint" solution. It serves both the Deploy workstream (Step A4) and the Optimize workstream (Step B2): it takes the audited ASR configuration and switches the named, AUDIT-VETTED rules to Block on the pilot ASR policy ('MDE ASR Rules - Pilot Ring - Block'). This is the only PowerShell that can change endpoint-security configuration, and it DEFAULTS TO A SAFE PREVIEW so you can see exactly which rules WOULD move to block before anything enforces. -Mode Hardening changes nothing — it prints the tamper/EDR-block-mode pre-change checklist. HOW TO RUN Preview which rules WOULD move to block (default — changes nothing): powershell -ExecutionPolicy Bypass -File .\MDEFULL-PromoteASR.ps1 `-PolicyName 'MDE ASR Rules - Pilot Ring - Block' -Rules '<guid1>','<guid2>'` Apply (explicit — required to make the change): `powershell -ExecutionPolicy Bypass -File .\MDEFULL-PromoteASR.ps1` -PolicyName 'MDE ASR Rules - Pilot Ring - Block' -Rules '<guid1>','<guid2>' -Apply Hardening pre-check (read-only): powershell -ExecutionPolicy Bypass -File .\MDEFULL-PromoteASR.ps1 -Mode Hardening Requires Microsoft.Graph with DeviceManagementConfiguration.ReadWrite.All (Apply) and Security Administrator / Endpoint Security Manager. Only run AFTER reviewing the audit evidence (MDEFULL-Report.ps1 -Section ASR / MDEFULL-Hunt.ps1 -Mode ASRAudit) and adding exclusions. EXPECTED RESULTS Preview: prints each rule that WOULD move audit -> block on the pilot ring. Apply: sets those rules to block. Validate with MDEFULL-Validate.ps1 -Check ASR and watch for false positives (MDEFULL-Hunt.ps1 -Mode ASRAudit -BlockedOnly) before promoting to production. Rollback: set the rule(s) back to audit in the policy, or restore the prior policy version. .NOTES White-label. DEFAULTS TO -WhatIf/preview. NEVER promote to block without an audit review first. Pilot ring only — expand ring-by-ring after each validates clean. AIR stays approval-gated regardless. > [CmdletBinding()] param([ValidateSet('ASR','Hardening')] [string]$Mode = 'ASR', [string]$PolicyName = 'MDE ASR Rules - Pilot Ring - Block', [string[]]$Rules, [switch]$Apply, [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

How to run: Preview which rules WOULD move to block (default — changes nothing): powershell -ExecutionPolicy Bypass -File .\MDEFULL-PromoteASR.ps1 `-PolicyName 'MDE ASR Rules - Pilot Ring - Block' -Rules '<guid1>','<guid2>'` Apply (explicit — required to make the change): `powershell -ExecutionPolicy Bypass -File .\MDEFULL-PromoteASR.ps1` -PolicyName 'MDE ASR Rules - Pilot Ring - Block' -Rules '<guid1>','<guid2>' -Apply Hardening pre-check (read-only): powershell -ExecutionPolicy Bypass -File .\MDEFULL-PromoteASR.ps1 -Mode Hardening Requires Microsoft.Graph with DeviceManagementConfiguration.ReadWrite.All (Apply) and Security Administrator / Endpoint Security Manager. Only run AFTER reviewing the audit evidence (MDEFULL-Report.ps1 -Section ASR / MDEFULL-Hunt.ps1 -Mode ASRAudit) and adding exclusions. EXPECTED RESULTS Preview: prints each rule that WOULD move audit -> block on the pilot ring. Apply: sets those rules to block. Validate with MDEFULL-Validate.ps1 -Check ASR and watch for false positives (MDEFULL-Hunt.ps1 -Mode ASRAudit -BlockedOnly) before promoting to production. Rollback: set the rule(s) back to audit in the policy, or restore the prior policy version. .NOTES White-label. DEFAULTS TO -WhatIf/preview. NEVER promote to block without an audit review first. Pilot ring only — expand ring-by-ring after each validates clean. AIR stays approval-gated regardless. > [CmdletBinding()] param([ValidateSet('ASR','Hardening')] [string]$Mode = 'ASR', [string]$PolicyName = 'MDE ASR Rules - Pilot Ring - Block', [string[]]$Rules, [switch]$Apply, [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

MDEFULL-Report.ps1

Purpose: Read-only reporting for the comprehensive "Microsoft Defender for Endpoint" solution. Summarizes what ASR rules WOULD block (from audit events), the noisiest repeating detections, onboarding/ sensor-health coverage, and the top software vulnerabilities (TVM exposure). With -SaveBaseline it records a "before" snapshot (alert volume, false-positive proxy, ASR block coverage, onboarding coverage, top-vuln count) so later runs show the before/after delta that makes the optimization measurable. Also reminds you to keep AIR approval-gated. HOW TO RUN Capture the "before" baseline (Optimize Step B1): powershell -ExecutionPolicy Bypass -File .\MDEFULL-Report.ps1 -Section All -SaveBaseline Later, show the delta vs a saved baseline: powershell -ExecutionPolicy Bypass -File .\MDEFULL-Report.ps1 -BaselinePath .\MDEFULL-baseline.json -Section accepts: ASR | Exposure | Noise | Coverage | Enablement | All (default All). Uses Microsoft.Graph.Authentication + Graph Security advanced hunting. Government cloud: -Environment. EXPECTED RESULTS An ASR audit summary (what block WOULD stop — review before promoting), the noisiest detections, coverage, the top TVM vulnerabilities, and drift versus the saved baseline. Downward noise/exposure and upward coverage = a healthy optimization trend. .NOTES White-label. Read-only. Use the ASR audit summary to add exclusions BEFORE moving to block. AIR automation must remain approval-gated until the SOC decides otherwise. > [CmdletBinding()] param([ValidateSet('ASR','Exposure','Noise','Coverage','Enablement','All')] [string]$Section = 'All', [switch]$SaveBaseline, [string]$BaselinePath = ".\MDEFULL-baseline.json", [int]$LookbackDays = 7, [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

How to run: Capture the "before" baseline (Optimize Step B1): powershell -ExecutionPolicy Bypass -File .\MDEFULL-Report.ps1 -Section All -SaveBaseline Later, show the delta vs a saved baseline: powershell -ExecutionPolicy Bypass -File .\MDEFULL-Report.ps1 -BaselinePath .\MDEFULL-baseline.json -Section accepts: ASR | Exposure | Noise | Coverage | Enablement | All (default All). Uses Microsoft.Graph.Authentication + Graph Security advanced hunting. Government cloud: -Environment. EXPECTED RESULTS An ASR audit summary (what block WOULD stop — review before promoting), the noisiest detections, coverage, the top TVM vulnerabilities, and drift versus the saved baseline. Downward noise/exposure and upward coverage = a healthy optimization trend. .NOTES White-label. Read-only. Use the ASR audit summary to add exclusions BEFORE moving to block. AIR automation must remain approval-gated until the SOC decides otherwise. > [CmdletBinding()] param([ValidateSet('ASR','Exposure','Noise','Coverage','Enablement','All')] [string]$Section = 'All', [switch]$SaveBaseline, [string]$BaselinePath = ".\MDEFULL-baseline.json", [int]$LookbackDays = 7, [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

MDEFULL-RunLab.ps1

Purpose: The enablement-workstream lab runner for the comprehensive "Microsoft Defender for Endpoint" solution. Lets a learner execute one hunting lab's query from PowerShell as well as the portal, so the whole cohort can run the same query and compare results. Carries a compact, self-contained copy of each hunting lab's query. Read-only — it only queries endpoint telemetry. HOW TO RUN pwsh -File .\MDEFULL-RunLab.ps1 -Lab 4 pwsh -File .\MDEFULL-RunLab.ps1 -Lab 5 -LookbackDays 7 -OutputCsv .\lab5.csv Prereqs: Microsoft.Graph; sign in with an account holding ThreatHunting.Read.All. Government cloud: pass -Environment. Verify table/column names / ASR ActionType names against your schema. EXPECTED RESULTS Matching rows to the console and, optionally, CSV. Labs 2/3/6/7 (incident graph, device timeline, Live Response, response-action walkthrough) are PORTAL exercises — not run here. The full commented pack is MDEFULL-Labs.kql; Lab 8 is the alert-only detection candidate ('MDE Lab Detection - Team - Pilot'). .NOTES White-label. Read-only. KQL via Graph runHuntingQuery. No credentials in code. > [CmdletBinding()] param([ValidateSet('1','4','5','8')] [string] $Lab = '1', [int] $LookbackDays = 1, [string] $OutputCsv, [ValidateSet('Global','USGov','USGovDoD')] [string]

$Environment = 'Global')

How to run: pwsh -File .\MDEFULL-RunLab.ps1 -Lab 4 pwsh -File .\MDEFULL-RunLab.ps1 -Lab 5 -LookbackDays 7 -OutputCsv .\lab5.csv Prereqs: Microsoft.Graph; sign in with an account holding ThreatHunting.Read.All. Government cloud: pass -Environment. Verify table/column names / ASR ActionType names against your schema. EXPECTED RESULTS Matching rows to the console and, optionally, CSV. Labs 2/3/6/7 (incident graph, device timeline, Live Response, response-action walkthrough) are PORTAL exercises — not run here. The full commented pack is MDEFULL-Labs.kql; Lab 8 is the alert-only detection candidate ('MDE Lab Detection - Team - Pilot'). .NOTES White-label. Read-only. KQL via Graph runHuntingQuery. No credentials in code. > [CmdletBinding()] param([ValidateSet('1','4','5','8')] [string] $Lab = '1', [int] $LookbackDays = 1, [string] $OutputCsv, [ValidateSet('Global','USGov','USGovDoD')] [string] $Environment = 'Global')

MDEFULL-Validate.ps1

Purpose: Read-only validation for the comprehensive "Microsoft Defender for Endpoint" solution. Confirms the area named by -Check against the Testing Guide pass/fail gates. Where a control is portal-only it prints the exact path to confirm manually; where it can be checked via advanced hunting (onboarding coverage, enablement-lab data, PoC benign-signal detections) it runs a read-only KQL query. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\MDEFULL-Validate.ps1 -Check Onboarding -Check accepts: Onboarding | EDR | ASR | Config | RBAC | Tuning | Enablement | PoC | All (default All). -DeviceName (optional) narrows the PoC/enablement telemetry checks. Government cloud: -Environment. EXPECTED RESULTS PASS/REVIEW/FAIL lines per check with the object names from the Implementation Guide, so you can confirm each Testing Guide gate before promoting ASR/block mode, expanding onboarding, operationalizing the labs, or calling a PoC conclusive. .NOTES White-label. Read-only. ASR must be audited before block. Lab ASR stays audit-only; the lab detection stays alert-only; PoC/AIR stays approval-gated. Verify P1/P2 feature availability. > [CmdletBinding()] param([ValidateSet('Onboarding','EDR','ASR','Config','RBAC','Tuning','Enablement','PoC','All')] [string]$Check = 'All', [string]$DeviceName, [int]$LookbackHours = 12, [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\MDEFULL-Validate.ps1 -Check Onboarding -Check accepts: Onboarding | EDR | ASR | Config | RBAC | Tuning | Enablement | PoC | All (default All). -DeviceName (optional) narrows the PoC/enablement telemetry checks. Government cloud: -Environment. EXPECTED RESULTS PASS/REVIEW/FAIL lines per check with the object names from the Implementation Guide, so you can confirm each Testing Guide gate before promoting ASR/block mode, expanding onboarding, operationalizing the labs, or calling a PoC conclusive. .NOTES White-label. Read-only. ASR must be audited before block. Lab ASR stays audit-only; the lab detection stays alert-only; PoC/AIR stays approval-gated. Verify P1/P2 feature availability. > [CmdletBinding()] param([ValidateSet('Onboarding','EDR','ASR','Config','RBAC','Tuning','Enablement','PoC','All')] [string]$Check = 'All', [string]$DeviceName, [int]$LookbackHours = 12, [ValidateSet('Global','USGov','USGovDoD')] [string]$Environment = 'Global')